

M³P-R1: Reinforcement Learning for Large Language Model Guided Multi-Modal Motion Planning via MIP Code Generation

Xingpeng Sun
Purdue University
United States
sun1223@purdue.edu

Zherong Pan
LightSpeed Studios
United States
zherong.pan.usa@gmail.com

Kai Cheng
Purdue University
United States
cheng753@purdue.edu

Xindi Tang
Purdue University
United States
tang666@purdue.edu

Syed Talha Bukhari
Purdue University
United States
bukhars@purdue.edu

Aniket Bera
Purdue University
United States
aniketbera@purdue.edu

Abstract: Multi-Modal Motion Planning (M³P) requires joint reasoning over continuous motions and discrete mode transitions, making it difficult to solve efficiently. For instance, a bipedal robot may walk to a target location and then use its arms to grasp an object. This scenario captures both mode transitions and continuous dynamics, yielding feasible paths that neither purely discrete nor continuous planners can handle. While Mixed-Integer Programming (MIP) offers a principled framework, constructing tractable formulations for non-convex problems is typically manual and domain-specific, especially in the *approximate, discretization-based* MIP regime needed for non-convex robotic tasks. We propose M³P-R1, a reinforcement learning method that fine-tunes large language models (LLMs) to decompose M³P tasks into MIP variables, constraints, and objectives. Instead of directly outputting answers—often prone to hallucination—the model generates executable Python code using MIP optimization libraries and constraint interfaces. This enables solver-backed execution for robust and verifiable solutions. Trained with an outcome-driven reward against the solver, M³P-R1 learns to compose modality-level discretization primitives and synthesize cross-modal coupling constraints, producing executable MIP programs for complex M³P tasks. We further introduce LLMM³P Bench, a benchmark of 1,316 problems spanning 5 single-modal and 8 multi-modal compositions, and use it to evaluate the proposed method. Experiments show that M³P-R1 achieves 91.7% single-modal and 80.9% multi-modal task success rate, improving over the strongest prompt-only LLM baseline (GPT-5.2) by 26.4 and 40.6 percentage points, respectively. M³P-R1 further transfers to real-world hardware at 87.5% success rate. These results demonstrate the effectiveness of solver-grounded LLM reasoning for reliable motion planning. Project page: <https://github.com/XingpengSun0/CoRL-26-M3P-R1>

Keywords: Multi-Modal Motion Planning, LLM for Robot Planning

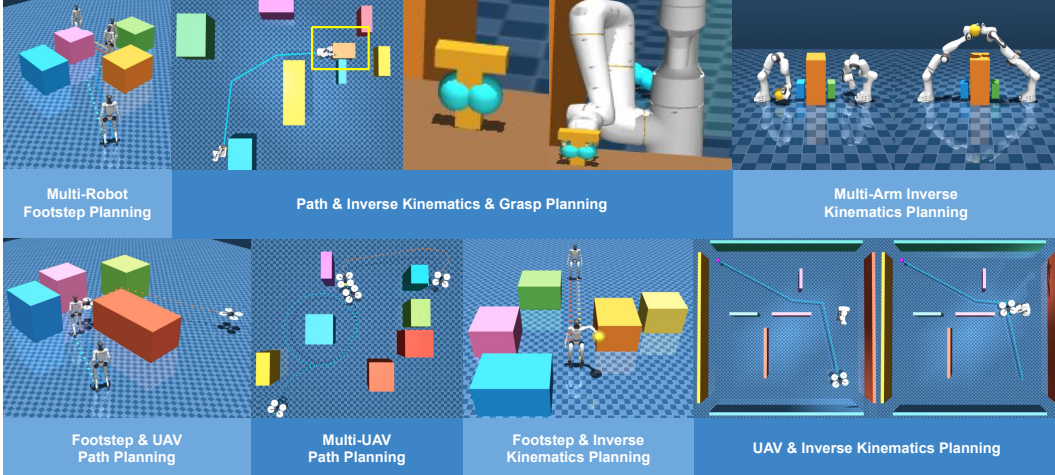


Figure 1: M³P-R1 enables unified planning for diverse M³P tasks: multi-robot footstep planning, unmanned aerial vehicle (UAV) navigation, inverse kinematics (IK), and grasping, as well as their compositional combinations. By generating solver-grounded MIP programs, the method produces feasible and robust trajectories across heterogeneous robotic domains. Trajectories are shown as dotted lines, with blue spheres on the T-shaped object marking predicted grasp contact points.

1 Introduction

In this paper, we address Multi-Modal Motion Planning (M³P) [1, 2], where robots must transition between distinct modes of operation (e.g., grasping, locomotion, navigation), as illustrated in Figure 1. As a specialized form of Task and Motion Planning (TAMP), M³P requires joint reasoning over discrete decisions and continuous motions, leading to a large hybrid decision space with tightly coupled symbolic and geometric constraints. While recent advances in TAMP [3, 4, 5, 6] and M³P [7, 8, 9] improve scalability, many approaches rely on domain-specific heuristics or assumptions such as local optimality [10, 11] or differentiability [4, 12], and often decouple task and motion planning, resulting in weak integration.

To address these limitations, unified formulations have been explored. Symbolic approaches based on the Planning Domain Definition Language (PDDL) [13, 14] enable efficient high-level reasoning but lack geometric grounding and depend on external motion planners [5]. In contrast, Mixed-Integer Programming (MIP) provides a principled framework for jointly modeling discrete and continuous variables within a single optimization problem. MIP has been successfully applied to grasp synthesis [15], caging [16], inverse kinematics [17], UAV planning [18, 19, 20], and footstep planning [21], enabling tighter integration of task–motion reasoning.

With the progress of LLMs, their use in robotics has expanded to interpreting high-level task descriptions and generating plans [22, 23, 24, 25]. However, prompt-based methods remain unreliable for long-horizon structured reasoning because their outputs are difficult to verify and prone to inconsistency. Prior LLM+P methods [26, 27, 28] improve reliability by producing symbolic plans, but remain limited to discrete abstractions. More broadly, Reinforcement Learning with Verifiable Rewards (RLVR) shows that grounding LLM reasoning in executable systems can improve reliability through objective, tool-based feedback. Motivated by this paradigm, we propose M³P-R1, where LLMs generate MIP formulations directly from natural-language M³P descriptions. We call this paradigm *LLM+MIP*, extending LLM+P [26] from symbolic PDDL planning to solver-grounded mixed-integer optimization. This setting requires semantic understanding, mathematical modeling, and program synthesis over hybrid decision spaces. MIP solvers provide feasibility and constraint-satisfaction signals for verification, but generating valid programs remains challenging due to non-convex planning and problem-specific discretization [29, 30, 31]. Building on recent advances in LLM mathematical reasoning [32, 33, 34], our experiments in Section 4 show that LLMs can learn such solver-grounded formulations.

To address this challenge, we propose a reinforcement fine-tuning (RFT) framework that trains LLMs to decompose robotic planning tasks into structured optimization components, including decision variables, constraints, and objective functions. By leveraging solver-derived feedback signals such as feasibility status and objective value, the model is directly optimized to generate valid MIP formulation code. This solver-in-the-loop training process enables the LLM to learn to decompose M³P tasks into various MIP constraints and objectives, and naturally extends to multi-modal motion planning scenarios. Note that we focus on *approximate*, discretization-based MIP formulations for non-convex robotic planning, in contrast to exact MIP/MILP formulations targeted by prior LLM-for-optimization work. To evaluate this framework, we introduce LLMM³P Bench, a benchmark dataset and evaluation protocol for LLM+MIP, consisting of diverse single- and multi-modal M³P instances. Experiments show that RFT enables M³P-R1 to outperform the strongest prompt-only LLM baseline (GPT-5.2) by 26.4/40.6 pp in single-/multi-modal task success rate.

Our core contributions are as follows:

1. **LLM+MIP method:** Building on prior LLM-to-optimization work that targets exact MIP/MILP formulations [35, 36, 37], we introduce a solver-grounded method that enables LLMs to generate executable *approximate* MIP formulations for M³P, bridging language-based reasoning with optimization-based planning over hybrid discrete–continuous decision spaces.
2. **RFT for structured decomposition:** We propose a RFT framework that leverages solver feedback to train LLMs to decompose robotic planning tasks into variables, constraints, and objectives, improving correctness, executability, and generalization to held-out task difficulties.
3. **Benchmark and empirical validation:** We present a new benchmark for LLM+MIP and demonstrate that RFT enables reliable generation of MIP programs across diverse single- and multi-modal planning tasks.

2 Related Work

TAMP, M³P, and MIP-based planning. Task and Motion Planning (TAMP) integrates high-level symbolic reasoning with continuous motion feasibility [5, 3], while Multi-Modal Motion Planning (M³P) focuses on transitions between locomotion, contact, and manipulation modes [1, 2]. Unified TAMP approaches, including Logic-Geometric Programming (LGP), jointly reason over discrete mode sequences and continuous geometric feasibility, while mixed-integer formulations such as ScottyActivity provide related unified optimization frameworks under convex continuous constraints. Thus, jointly coupling task-level choices with continuous feasibility is not itself our novelty. Mixed-Integer Programming (MIP) has also been applied to grasping, caging, inverse kinematics, UAV navigation, multi-robot planning, and footstep planning [15, 16, 17, 18, 19, 20, 21]. In contrast to these human-specified planning formulations, our focus is on generating executable optimization programs directly from natural-language task descriptions, including approximate formulations for non-convex geometric constraints using IRIS-style convex decomposition. Sampling-based and trajectory-optimization methods such as RRT*, BIT*, PRM, and TrajOpt [38, 39, 40, 41] handle continuous motion but typically decouple geometric planning from discrete mode reasoning. PDDL-Stream [42] connects symbolic planning with continuous feasibility through hand-designed streams, but requires a manually specified symbolic model and continuous samplers.

LLMs for planning. LLMs have been used for robotic planning through direct prompting, symbolic translation, and code generation [22, 23, 24, 25]. Methods such as ReAct [43] improve interactive reasoning but lack formal optimization grounding. LLM+P-style approaches translate language into PDDL or related symbolic representations [26, 27, 28, 44], while Code as Policies and ProgPrompt generate executable robot programs [45, 46]. Prior work has also explored richer language and multimodal interfaces for robot planning. TrustNavGPT models uncertainty in audio-guided LLM navigation [47], while Beyond Text incorporates vocal cues into LLM decision making for robot navigation [48]. In inspection planning, Sun et al. [49] use a VLM to interpret language-specified inspection goals and combine VLM-selected viewpoints with TSP and trajectory optimization to generate executable paths. These methods improve language-conditioned planning but rely on pre-

defined planning or optimization pipelines rather than synthesizing the underlying optimization formulation itself. Similarly, waypoint-based approaches that combine LLM outputs with RRT*, BIT*, or TrajOpt still separate semantic decomposition from continuous optimization and therefore struggle with tightly coupled M³P constraints.

Foundation models have also been combined with classical geometric optimization outside robot planning. For example, Sun et al. [50] use a VLM to interpret natural-language deformation instructions, select deformation handles, and couple these predictions with a classical handle-based mesh deformation algorithm. Such systems demonstrate the benefit of using foundation models as semantic interfaces to structured geometric solvers, but the downstream optimization model remains fixed and manually designed. In contrast, our goal is to train an LLM to construct the optimization program itself, including task-specific variables, objectives, constraints, discretization primitives, and cross-modal coupling constraints.

LLMs for MIP and solver-grounded reinforcement fine-tuning. Recent work has explored LLMs for optimization, including diagnosing infeasible models, translating natural language into MIP scripts, and learning foundation models for MILP [51, 52, 53, 35, 36, 37]. However, these methods primarily target exact MIP or operations-research-style MILP instances. In contrast, robotic M³P requires approximate MIP formulations with geometry-specific discretization choices. Reinforcement fine-tuning methods such as RLHF, PPO, DPO, and GRPO [54, 55, 56, 57] show that outcome-based rewards can improve structured reasoning, especially when external tools provide verifiable feedback. Solver-grounded and tool-integrated LLM systems further demonstrate the value of executable verification [58, 59, 60].

Unlike classical TAMP and MIP-based M³P methods [5, 15, 21] and prior language/VLM-guided planning systems [49, 50], which rely on human-specified models or fixed downstream optimizers, M³P-R1 maps natural-language tasks directly to executable optimization programs by generating task-specific variables, constraints, objectives, and cross-modal couplings. Unlike LLM planning and program-generation methods that use execution or solver feedback mainly for test-time refinement [26, 45, 46], we use solver feedback as a reinforcement fine-tuning signal, enabling single-pass program generation at inference.

3 Method

Our pipeline is shown in Figure 2. To translate M³P descriptions into executable MIP formulations, we expose an existing MIP modeling language as an LLM-callable Application Programming Interface (API). We use CVXPY [61], a high-level optimization package supporting multiple backend solvers, as the modeling interface, with Gurobi [62] as the default solver.

Approximate MIP for M³P requires problem-specific discretization, such as Iterative Regional Inflation by Semidefinite Programming (IRIS) [30] and related methods [29, 31], to approximate non-convex constraints with disjoint convex representations. Since deriving such discretizations from scratch is beyond our scope, we provide CVXPY-integrated discretization primitives as LLM-callable APIs. The library contains only 23 modality-level functions, reused unchanged across all five single-modal and eight multi-modal task families. It contains no multi-modal template or coupling primitive: cross-modal interfaces such as meeting points, contact locations, shared states, and reachability constraints must be generated by the LLM. Thus, multi-modal planning still requires the LLM to synthesize a joint MIP formulation by composing single-modality primitives and adding coupling constraints, such as shared meeting points, contact locations, reachability constraints, and synchronized intermediate states. The fine-tuned LLM therefore learns to generate executable code that uses these primitives while constructing task-specific MIP constraints and objectives for M³P. The system prompt and representative `<think>/<code>` output are shown in Appendix K; the API specification, MIP formulations, and data samples are provided in Appendix F, G, and J. We introduce the dataset and GRPO training below.

LLMM³P Bench We introduce LLMM³P Bench, a dataset of 1,316 sampled M³P problems covering five single-modal tasks and eight multi-modal compositions that integrate locomotion, manipulation, and aerial navigation (Appendix A). Each sample consists of: (i) a natural language task

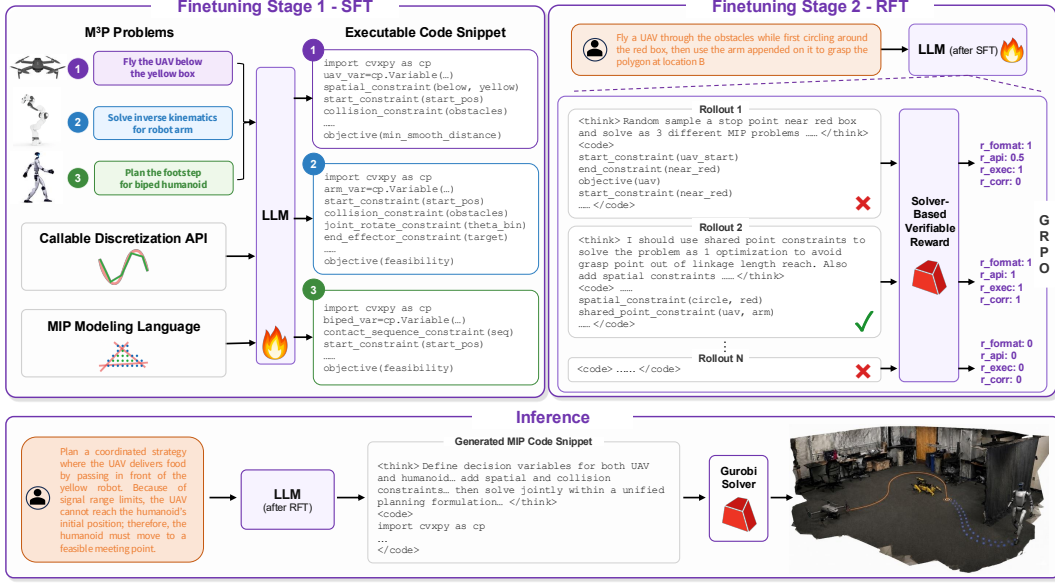


Figure 2: M³P-R1 pipeline. The model is first supervised to generate structured MIP programs via a discretization API, then refined with GRPO using solver-verifiable rewards (format, API coverage, execution, and correctness). At inference, it maps natural language tasks to executable MIP code with intermediate reasoning, which is solved to yield feasible multi-modal plans.

description, including an obstacle map specifying the environment layout and object boundaries, and (ii) fully executable MIP trajectory code. A statistical summary is shown in Figure 3. In total, LLMM³P Bench comprises 16.8K lines of code (LoC), with an average of 138 LoC per sample (range: 63–260), and 23K function calls, averaging 138 LoC and 19.3 API calls per program (range: 11–36). These statistics highlight both the diversity and structural complexity of the dataset, making it a challenging benchmark for evaluating whether LLMs can generalize to generate correct, executable, and scalable MIP formulations for robotic motion planning tasks. Of the 1,316 samples, 100 are used for supervised fine-tuning (SFT), 1,040 for GRPO training (including the SFT subset), and 276 for evaluation. We detail the dataset collection pipeline in Appendix B. We will release the LLMM³P Bench dataset, training code, and model checkpoints upon acceptance.

Training Pipeline Let f denote a generic base LLM with parameters θ_0 adapted into policy π_θ ; given a mission specification x , its task environment $\mathcal{E}_x$ (workspace bounds, obstacles, start and goal states; full tuple in Appendix B), and a predefined MIP discretization tool library $\mathcal{T} = \{t_k\}$, the policy $\pi_\theta(y|x)$ generates an executable program y that composes API calls $t_k \in \mathcal{T}$ with variable, constraint, and objective definitions. The program defines a MIP instance executed by solver $\mathcal{S}$, producing $(\tau, \mathcal{L}) = \mathcal{S}(y, \mathcal{E}_x)$, where τ is the trajectory and $\mathcal{L}$ is the API execution log; learning therefore optimizes π_θ to both select modality-level tools and compose them with newly generated variables, constraints, and objectives into a unified MIP formulation through two stages: SFT on single-modal demonstrations to obtain $\pi_{\theta_{\text{SFT}}}$, followed by GRPO refinement with solver-verifiable rewards, instantiating RLVR with deterministic solver feedback.

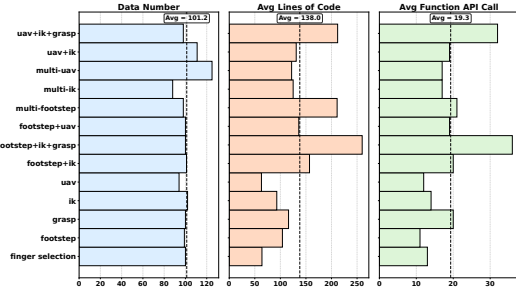


Figure 3: Dataset statistics. Left: total samples per modality; Middle: LoC distribution; Right: function-call distribution. Bars are grouped by modality (single-modal vs. multi-modal compositions); single-modal tasks are smaller, while multi-modal compositions are more complex.

In Stage 1, we perform supervised fine-tuning on single-modal demonstrations $\mathcal{D}_{\text{single}}$ by maximizing the log-likelihood objective:

$$\mathcal{J}_{\text{SFT}}(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{single}}} [\log \pi_{\theta}(y|x)].$$

This yields a bootstrapped policy $\pi_{\theta_{\text{SFT}}}$, whose output $y \sim \pi_{\theta_{\text{SFT}}}(\cdot|x)$ is a syntactically valid program that follows the required format, but is not yet optimized for solver success or task feasibility.

In Stage 2, starting from $\pi_{\theta_{\text{SFT}}}$, we optimize tool usage and solution quality via Group Relative Policy Optimization (GRPO) [57]. GRPO computes advantages by normalizing rewards within each sampled group, eliminating the need for a learned value function. For each x , we sample a group of G programs $\{y_i\}_{i=1}^G \sim \pi_{\theta}(\cdot|x)$ (group size G), execute them with $\mathcal{S}$, let τ_i be the trajectory component of $\mathcal{S}(y_i, \mathcal{E}_x)$, and compute reward $r_i = \mathcal{R}_{\text{rew}}(x, y_i, \tau_i)$:

$$\mathcal{R}_{\text{rew}}(x, y, \tau) = \lambda_1 R_{\text{format}}(y) + \lambda_2 R_{\text{API}}(x, y) + \lambda_3 R_{\text{exec}}(y) + \lambda_4 R_{\text{corr}}(x, y, \tau),$$

where $\lambda_1, \dots, \lambda_4 \geq 0$ are scalar weighting coefficients balancing the four reward terms (values reported in Appendix H.1, Table 14). The format reward is defined as the indicator: $R_{\text{format}}(y) = \mathbb{1}[y \text{ matches the } \langle \text{think} \rangle \dots \langle \text{code} \rangle \dots \langle \text{code} \rangle \text{ schema}] \in \{0, 1\}$, enforcing the required structured output schema by checking the presence and proper closure of the reasoning $\langle \text{think} \rangle \dots \langle \text{think} \rangle$ and code $\langle \text{code} \rangle \dots \langle \text{code} \rangle$ blocks. The execution reward is the indicator (with the fixed solver $\mathcal{S}$ and environment $\mathcal{E}_x$ treated as hyper-parameters): $R_{\text{exec}}(y) = \mathbb{1}[y_{\text{code}} \text{ executes successfully on } \mathcal{S}] \in \{0, 1\}$, which verifies that the generated code y_{code} —the content inside the $\langle \text{code} \rangle \dots \langle \text{code} \rangle$ block of y —is syntactically valid and executable in Python, when executed in $\mathcal{E}_x$ against the fixed solver $\mathcal{S}$. To teach the model to use the correct MIP discretization APIs, let $\mathcal{A}(x) \subseteq \mathcal{T}$ denote the required discretization APIs and $\hat{\mathcal{A}}(y) \subseteq \mathcal{T}$ those invoked in y ; we define: $R_{\text{API}}(x, y) = \frac{|\hat{\mathcal{A}}(y) \cap \mathcal{A}(x)|}{|\mathcal{A}(x)|}$. This coverage-based reward provides a smoother optimization signal than binary correctness, assigning partial credit to incomplete but valid API usage, thereby improving training stability. The required API set is used only during training as an auxiliary reward and is obtained from the executable expert traces in LLMM³P Bench; it is not provided at inference time. For trajectory correctness, let $\tau = (s_t)_{t=1}^T$ denote the solver-generated trajectory of length T , with $s_t \in \mathcal{X}$ for a task-dependent state space $\mathcal{X}$ (concrete instantiation per modality in Appendix G); let $\text{stat}(y, \mathcal{E}_x)$ denote the solver status induced by executing y in $\mathcal{E}_x$, and let $\text{stat}^*(x)$ denote the ground-truth solver status for task x . Here Φ_{init} and Φ_{goal} verify that all agents’ start and goal states match the mission specification x , and Φ_{dyn} enforces that the trajectory satisfies all spatial and task constraints, including collision-free motion for each agent and consistent coordination at shared states for multi-agent tasks. Each Φ_* may also use the environment $\mathcal{E}_x$; we treat $\mathcal{E}_x$ as packaged inside x . We define $R_{\text{corr}}(x, y, \tau) = \mathbb{1}[\text{stat}(y, \mathcal{E}_x) = \text{stat}^*(x) \wedge \Phi_{\text{init}}(x, \tau) = 1 \wedge \Phi_{\text{goal}}(x, \tau) = 1 \wedge \Phi_{\text{dyn}}(x, \tau) = 1]$. Trajectory correctness is evaluated by executing the generated program with the solver and checking task constraints, rather than comparing against a fixed reference trajectory. This reward assigns 1 only when the generated program yields a solver-consistent and fully valid trajectory, and 0 otherwise. Rewards are normalized within each group: with $\bar{r}$ and σ_r denoting the mean and standard deviation of $\{r_i\}_{i=1}^G$, we define the group-normalized advantage as $\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r}$. Let $\pi_{\theta_{\text{old}}}$ denote the behavior policy used to generate each sampled group. We optimize the standard clipped GRPO objective

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}_{x \sim \mathcal{D}_{\text{GRPO}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{T_i} \sum_{t=1}^{T_i} \min \left(\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right],$$

where $\rho_{i,t} = \frac{\pi_{\theta}(y_{i,t}|h_{i,t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|h_{i,t})}$, $h_{i,t}$ denotes the generation history preceding token t , T_i is the length of program y_i , and ϵ is the clipping coefficient. In our implementation, we use no additional KL-divergence penalty ($\beta = 0$). The solver $\mathcal{S}$, training set $\mathcal{D}_{\text{GRPO}}$, and reward functions $\{R_{\text{format}}, R_{\text{API}}, R_{\text{exec}}, R_{\text{corr}}\}$ are fixed during training. This objective refines $\pi_{\theta_{\text{SFT}}}$ into a policy that learns to compose MIP discretization APIs into executable formulations whose solver outputs satisfy the task specification.

Table 2: Comparison with symbolic/prompt (left) and sampling-based (right) baselines. Values are reported as mean $\pm$ standard deviation over three runs. *SR (%)* is task-level success rate (API/Exec/SR metrics defined in Table 1). *Time* denotes end-to-end planning time (LLM inference + solver solve) in seconds. *Qual* is the ratio of path length and trajectory smoothness relative to our method (1.00/1.00), computed only on instances where both Ours and the baseline succeed; lower is better. PDDLStream operates on pre-parsed (gold) symbolic input rather than free-form language (see Appendix H.2).

Symbolic & Prompt Baselines						Sampling-based Baselines					
Method	Single			Multi			Method	Single		Multi	
	SR	Time	Qual	SR	Time	Qual		Path	Smooth	Path	Smooth
Ours	91.7 $\pm$ 0.6	20.2 $\pm$ 1.1	1.00/1.00	80.9 $\pm$ 2.3	99.7 $\pm$ 7.8	1.00/1.00	Ours	1.00 $\pm$ 0.05	1.00 $\pm$ 0.04	1.00 $\pm$ 0.17	1.00 $\pm$ 0.22
PDDLStream [42]	95.0 $\pm$ 3.5	18.1 $\pm$ 2.4	1.21/1.36	57.6 $\pm$ 5.8	101.5 $\pm$ 8.2	1.42/1.29	LLM+RRT* [38]	0.95 $\pm$ 0.11	1.48 $\pm$ 0.19	1.36 $\pm$ 0.14	1.73 $\pm$ 0.27
LLM+P [26]	85.9 $\pm$ 5.7	33.8 $\pm$ 4.9	1.39/1.48	39.4 $\pm$ 6.3	118.6 $\pm$ 11.5	1.78/1.61	LLM+BIT* [39]	0.93 $\pm$ 0.08	1.46 $\pm$ 0.21	1.24 $\pm$ 0.12	1.57 $\pm$ 0.42
ReAct [43]	69.8 $\pm$ 8.1	64.7 $\pm$ 12.2	2.51/2.44	24.6 $\pm$ 6.9	149.8 $\pm$ 20.3	3.11/1.89	LLM+TrajOpt [41]	1.09 $\pm$ 0.14	1.34 $\pm$ 0.11	1.18 $\pm$ 0.16	1.34 $\pm$ 0.08

4 Experiment

Our evaluation focuses on answering four questions: (i) *Can LLMs correctly generate code that uses the provided API functions to assemble a MIP formulation that directly solves an M³P task described in natural language?* (ii) *How does M³P-R1 compare to symbolic and sampling-based planning baselines?* (iii) *What is the contribution of each training stage and reward component?* (iv) *Does the approach transfer to real-world hardware?* We use Qwen3-8B as our base model. Detailed training parameters are provided in Appendix H.1.

Experimental Setup. Metrics.

We report three complementary metrics throughout: **API (%)** measures structured output correctness (how often the generated code uses the discretization APIs with valid signatures); **Exec (%)** measures whether the generated program executes successfully on the solver; and **SR (%)** measures full task-level correctness against the ground-truth solver status and trajectory constraints (Appendix H.1).

LLM-baseline evaluation protocol. All prompt-only LLM baselines, as well as the LLM component of LLM+P, LLM+RRT*, LLM+BIT*, and LLM+TrajOpt, use **GPT-5.2** as their underlying LLM, with sampling temperature $T_{\text{samp}} = 0.6$ and one in-context example for format alignment. ReAct uses Qwen3-8B as the backbone, with the same API handbook provided in context. Parse failures are retried once before being recorded as a failure. The full protocol is documented in Appendix H.2. PDDLStream operates on pre-parsed (gold) symbolic input rather than free-form language.

Quantitative analysis In Table 1, we compare M³P-R1 with proprietary LLMs (GPT-5.2, Gemini-2.5-Flash), open-source LLMs of different scales (Qwen3, LLaMA3.1, Gemma3, 4B–30B), and a coding-specialized model (Qwen3-Coder-Next). We report API/Exec/SR separately for *single-modal* tasks (UAV, IK, footstep, grasp, and finger selection) and *multi-modal* compositions (e.g., UAV+IK, multi-UAV, multi-IK, multi-footstep, and higher-order combinations), following the protocol in Appendix H.2. All baselines receive one in-context example for format alignment. Although this improves API and execution rates, prompt-only methods remain unreliable on task correctness, especially for multi-modal tasks (40.3% SR), motivating RFT with solver-verifiable rewards. In Table 2, we compare against six symbolic and sampling-based baselines. We report success rate (SR), end-to-end planning time (LLM inference + solver solve), and solution

Table 1: Average performance for LLMs across single-modal and multi-modal settings over three runs. Values are mean $\pm$ standard deviation.

Model	Single-modal			Multi-modal		
	API (%)	Exec (%)	SR (%)	API (%)	Exec (%)	SR (%)
Ours	98.0 $\pm$ 1.7	94.3 $\pm$ 1.2	91.7 $\pm$ 0.6	94.5 $\pm$ 0.3	88.8 $\pm$ 1.8	80.9 $\pm$ 2.3
GPT-5.2	89.3 $\pm$ 4.0	88.3 $\pm$ 4.5	65.3 $\pm$ 6.5	83.9 $\pm$ 3.1	60.0 $\pm$ 3.7	40.3 $\pm$ 4.0
Gemini 2.5 Flash	86.3 $\pm$ 6.0	56.3 $\pm$ 4.6	49.0 $\pm$ 11.0	84.8 $\pm$ 6.3	50.9 $\pm$ 7.7	35.8 $\pm$ 6.2
Qwen3-30B	68.7 $\pm$ 10.5	81.7 $\pm$ 10.5	56.7 $\pm$ 9.5	38.8 $\pm$ 6.2	34.7 $\pm$ 6.8	30.3 $\pm$ 6.0
Gemma3-12B	81.7 $\pm$ 7.6	39.7 $\pm$ 12.0	35.7 $\pm$ 7.5	35.8 $\pm$ 6.8	28.6 $\pm$ 6.0	14.2 $\pm$ 5.1
Qwen3-8B	67.0 $\pm$ 11.8	76.0 $\pm$ 6.1	58.3 $\pm$ 9.8	60.0 $\pm$ 8.2	33.1 $\pm$ 7.1	25.4 $\pm$ 5.2
LLaMA3.1-8B	52.7 $\pm$ 12.5	75.7 $\pm$ 7.5	52.7 $\pm$ 7.5	32.4 $\pm$ 5.7	38.8 $\pm$ 6.2	27.7 $\pm$ 6.5
Qwen3-Coder-Next	52.3 $\pm$ 8.7	83.3 $\pm$ 9.3	51.7 $\pm$ 9.2	42.0 $\pm$ 6.8	43.9 $\pm$ 8.2	33.9 $\pm$ 7.1
Qwen3-4B	34.3 $\pm$ 21.4	27.3 $\pm$ 8.3	23.0 $\pm$ 8.2	21.2 $\pm$ 5.4	15.0 $\pm$ 4.8	10.8 $\pm$ 4.0
Gemma3-4B	49.0 $\pm$ 6.2	4.7 $\pm$ 5.0	4.3 $\pm$ 5.1	21.8 $\pm$ 4.6	7.0 $\pm$ 4.3	1.9 $\pm$ 2.0

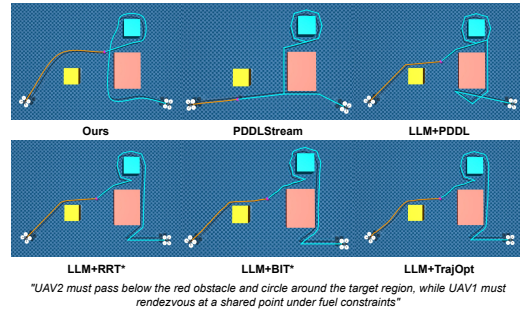


Figure 4: Comparison of multi-UAV path planning. UAV trajectories are shown in orange (UAV1) and blue (UAV2). Our method generates smoother and shorter trajectories that better satisfy spatial requirements. The bottom three sampling-based methods do not complete a full circle around the blue box, and therefore fail the spatial requirement.

quality (Qual). Qual is evaluated only on successful single- and multi-UAV instances where both methods succeed. For a trajectory $\tau = (s_t)_{t=1}^T$, $s_t \in \mathbb{R}^3$, smoothness is measured by discrete curvature $\sum_{t=2}^{T-1} \|s_{t+1} - 2s_t + s_{t-1}\|$. Qual reports path length and smoothness normalized by our method (1.00/1.00), where lower is better. PDDLStream [42] uses symbolic PDDL with black-box streams for continuous feasibility but struggles with tightly coupled multi-modal tasks. LLM+P [26] generates PDDL instances under fixed symbolic abstractions, while ReAct [43] performs ungrounded iterative prompting. LLM+RRT* [38], LLM+BIT* [39], and LLM+TrajOpt [41] use LLM-proposed waypoints or shared points followed by classical planners, and are limited to UAV settings.

As shown in Table 2, M³P-R1 achieves the best overall performance, with high SR and low runtime across both settings. PDDLStream attains higher single-modal SR (95.0% vs. 91.7%) because it operates on fully grounded symbolic specifications without language understanding, but degrades substantially on multi-modal tasks where discrete-continuous coordination is tightly coupled. LLM+P and ReAct underperform due to brittle symbolic abstractions and ungrounded reasoning. RRT* and BIT* produce shorter single-UAV paths, but our method is more effective in multi-UAV settings by jointly optimizing meeting points and smoothness, yielding shorter and smoother trajectories. These results show that solver-grounded MIP formulations better capture continuous objectives than sampling-based pipelines, and that composing discretization APIs with solver-verifiable rewards enables robust and scalable planning across diverse M³P tasks.

To test whether our gains can be explained simply by access to expert optimization modules, we construct an Expert-MIP baseline in which an LLM selects fixed single-modal MIP modules and predicts only the cross-modal interface variables required to compose them. As shown in Table 4, the expert modules achieve 100.0% success on single-modal tasks with either Qwen3-8B or GPT-5.2. However, multi-modal success drops to 55.6% and 63.2%, respectively, compared with 80.9% for M³P-R1. Predicted meeting points, contact locations, or intermediate states can make downstream IK, grasp, or motion-planning subproblems infeasible, indicating that access to expert modules alone does not resolve the cross-modal coupling learned through solver-grounded reinforcement fine-tuning.

In Table 3, GRPO-200 (GRPO for 200 steps) improves over SFT-only training, while removing SFT degrades performance, highlighting that SFT is beneficial for tool-augmented RL [63]. The *Ours* (800 steps) row reports the final M³P-R1 model as a reference upper bound, while all four reward ablations (w/o $R_{\bullet}$, with $\bullet \in \{\text{format, API, exec, corr}\}$) share the matched 200-step GRPO budget and should be compared directly against the *GRPO-200* row. Additional held-out generalization studies are provided in Appendix C.4 and C.5, including unseen modality compositions and out-of-distribution (OOD) shifts in obstacle density, agent count, and spatial-relation complexity. M³P-R1 maintains strong success under these shifts, e.g., 85.7/70.5 SR on Obstacle-OOD, 89.9/75.9 SR on Agent-OOD, and 87.4/76.8 SR on Spatial-OOD for single-/multi-modal tasks, suggesting that it learns reusable MIP-programming abstractions and cross-modal constraint patterns rather than memorizing fixed templates. To further control for training budget, we rerun all reward-component ablations for the full 800 GRPO steps used by the final model. At this matched budget, removing R_{corr} produces the largest degradation, reducing SR from 84.8% to 50.3%, while removing R_{exec} and R_{API} lowers SR to 70.0% and 73.5%, respectively. In contrast, removing R_{format} has a smaller

Table 3: Ablation on training stages and reward terms. Results are data-count-weighted averages over all single- and multi-modal tasks, reported as mean $\pm$ standard deviation over three runs. *Ours* (800 steps) is the final M³P-R1 reference from Table 1; reward ablations are compared against the matched *GRPO-200* checkpoint.

Model	API(%)	Exec(%)	SR(%)
Ours (800 steps)	95.3 $\pm$ 0.4	90.6 $\pm$ 1.8	84.8 $\pm$ 1.6
<i>Training stages</i>			
GRPO-200	89.5 $\pm$ 2.5	62.7 $\pm$ 2.5	58.7 $\pm$ 3.3
GRPO-200 w/o SFT	75.7 $\pm$ 3.3	56.9 $\pm$ 2.9	53.3 $\pm$ 2.9
SFT only	80.8 $\pm$ 2.2	64.5 $\pm$ 2.5	56.3 $\pm$ 3.1
<i>Reward terms</i>			
w/o R_{format}	58.8 $\pm$ 2.7	27.2 $\pm$ 3.3	29.7 $\pm$ 2.9
w/o R_{API}	69.9 $\pm$ 3.3	53.9 $\pm$ 2.9	46.4 $\pm$ 2.9
w/o R_{exec}	85.7 $\pm$ 2.0	43.5 $\pm$ 3.6	41.1 $\pm$ 3.4
w/o R_{corr}	82.0 $\pm$ 2.7	70.3 $\pm$ 2.5	40.6 $\pm$ 4.0

Table 4: Expert-MIP control. The LLM selects fixed expert single-modal MIP modules and predicts the cross-modal interface variables required to compose.

Method	Single-modal SR (%)	Multi-modal SR (%)
Expert-MIP + Qwen3-8B	100.0	55.6
Expert-MIP + GPT-5.2	100.0	63.2
M ³ P-R1	91.7	80.9

effect at convergence, yielding 78.4% SR. These results confirm that solver-verified task correctness provides the strongest training signal, while execution and API rewards remain important for learning reliable executable formulations.

Qualitative analysis Figure 4 compares our method with representative planning baselines. Symbolic planners (e.g., PDDL) and sampling-based methods (e.g., RRT-style) often produce inefficient or zigzag trajectories due to discrete abstractions or lack of global optimality, while trajectory optimization (TrajOpt) improves smoothness but remains sub-optimal under complex multi-modal constraints. We further validate our approach in physics simulation (MuJoCo), as shown in Figure 18 and 19 (Appendix D). Across both single- and multi-modal tasks, our method generates feasible, collision-free, and coordinated trajectories that satisfy geometric, kinematic, and task-level constraints. Detailed per-modality MIP formulations are provided in Appendix G (see Appendix G.2 for our novel spatial constraints and G.4 for the grasp formulation).

Real-world robotic experiment We validate M³P-R1 on real hardware using a Franka Panda robot arm and DJI Tello drones across 40 scenarios, achieving an overall success rate of 87.5% (35/40). Failures are primarily attributed to sim-to-real mismatch (Appendix E). The Franka Panda operates at 1 kHz over Ethernet, while the Tello communicates via 2.4 GHz WiFi. As shown in Figure 5, our method successfully executes both (a) IK and (b) multi-UAV tasks, demonstrating that M³P-R1 transfers reliably from simulation to real-world deployment.

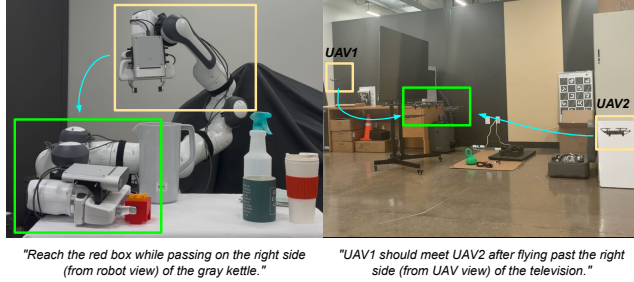


Figure 5: Real-world execution. (a) The robot arm performs obstacle-aware IK by following a constrained trajectory. (b) Two UAVs coordinate to satisfy spatial constraints and rendezvous. These results demonstrate reliable execution of solver-generated plans in real environments.

5 Conclusion, Limitations, and Future Work

Conclusion. We introduce M³P-R1, a novel method for LLM-guided robot motion planning. Our approach generates solver-grounded MIP formulations, enabling joint reasoning over discrete and continuous decision spaces. Crucially, through GRPO with solver-verifiable rewards, the model learns to compose discretization APIs and construct MIP decompositions that solve M³P tasks, rather than relying on memorized patterns. We demonstrate that M³P-R1 effectively handles diverse single- and multi-modal M³P tasks, a subset of TAMP. Empirically, M³P-R1 reaches 91.7%/80.9% single-/multi-modal success rate—a 26.4/40.6 pp gain over the strongest prompt-only LLM baseline (GPT-5.2)—and transfers to real Franka+Tello hardware at 87.5% success.

Limitation. Despite promising results, our work has several limitations that motivate future research. First, due to the inherent limitations of MIP formulation, our pipeline is primarily designed for static environments and does not yet support dynamic scenes with moving obstacles or time-varying constraints. Second, our study focuses on MIP-based robot planning problems already developed by human experts. A promising future direction is to enable LLMs to propose, formulate, and solve previously unmodeled M³P problems, such as whole-body control and more general contact-rich robotic behaviors.

Acknowledgments

We thank the anonymous reviewers for their valuable feedback and the IDEAS Lab at Purdue University for providing robotic hardware support.

References

- [1] K. Hauser, V. Ng-Thow-Hing, and H. Gonzalez-Baños. Multi-modal motion planning for a humanoid robot manipulation task. In *Robotics Research: The 13th International Symposium ISRR*, pages 307–317. Springer, 2010.
- [2] K. Hauser and J.-C. Latombe. Multi-modal motion planning in non-expansive spaces. *The International Journal of Robotics Research*, 29(7):897–915, 2010.
- [3] N. T. Dantam, Z. K. Kingston, S. Chaudhuri, and L. E. Kavraki. Incremental task and motion planning: A constraint-based approach. In *Robotics: Science and systems*, volume 12, page 00052. Ann Arbor, MI, USA, 2016.
- [4] M. A. Toussaint, K. R. Allen, K. A. Smith, and J. B. Tenenbaum. Differentiable physics and stable modes for tool-use and manipulation planning. 2018.
- [5] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez. Integrated task and motion planning. *Annual review of control, robotics, and autonomous systems*, 4(1):265–293, 2021.
- [6] Z. Jiao, Y. Niu, Z. Zhang, S.-C. Zhu, Y. Zhu, and H. Liu. Sequential manipulation planning on scene graph. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8203–8210. IEEE, 2022.
- [7] Z. Kingston, A. M. Wells, M. Moll, and L. E. Kavraki. Informing multi-modal planning with synergistic discrete leads. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3199–3205. IEEE, 2020.
- [8] L. L. Beyer, N. Balabanska, E. Tal, and S. Karaman. Multi-modal motion planning using composite pose graph optimization. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9981–9987. IEEE, 2021.
- [9] Z. Kingston and L. E. Kavraki. Scaling multimodal planning: Using experience and informing discrete search. *IEEE Transactions on Robotics*, 39(1):128–146, 2022.
- [10] Z. Zhao, S. Cheng, Y. Ding, Z. Zhou, S. Zhang, D. Xu, and Y. Zhao. A survey of optimization-based task and motion planning: From classical to learning approaches. *IEEE/ASME Transactions on Mechatronics*, 2024.
- [11] Z. Manchester and S. Kuindersma. Variational contact-implicit trajectory optimization. In *Robotics Research: The 18th International Symposium ISRR*, pages 985–1000. Springer, 2019.
- [12] J. Envall, R. Poranne, and S. Coros. Differentiable task assignment and motion planning. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2049–2056. IEEE, 2023.
- [13] M. Helmert. The fast downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006.
- [14] W. Piotrowski, A. Perez, and S. Grover. Nyx: Planning for emerging problems with pddl+ and beyond. 2024.
- [15] M. Liu, Z. Pan, K. Xu, and D. Manocha. New formulation of mixed-integer conic programming for globally optimal grasp planning. *IEEE Robotics and Automation Letters*, 5(3):4663–4670, 2020.

- [16] B. Aceituno-Cabezas, H. Dai, and A. Rodriguez. A convex-combinatorial model for planar caging. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2378–2385. IEEE, 2019.
- [17] H. Dai, G. Izatt, and R. Tedrake. Global inverse kinematics via mixed-integer convex optimization. *The International Journal of Robotics Research*, 38(12-13):1420–1441, 2019.
- [18] R. Deits and R. Tedrake. Efficient mixed-integer planning for uavs in cluttered environments. In *2015 IEEE international conference on robotics and automation (ICRA)*, pages 42–49. IEEE, 2015.
- [19] J. Yu and S. M. LaValle. Planning optimal paths for multiple robots on graphs. In *2013 IEEE International Conference on Robotics and Automation*, pages 3612–3617. IEEE, 2013.
- [20] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake. Motion planning around obstacles with convex optimization. *Science robotics*, 8(84):eadf7843, 2023.
- [21] R. Deits and R. Tedrake. Footstep planning on uneven terrain with mixed-integer convex optimization. In *2014 IEEE-RAS international conference on humanoid robots*, pages 279–286. IEEE, 2014.
- [22] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- [23] Y. Ding, X. Zhang, C. Paxton, and S. Zhang. Task and motion planning with large language models for object rearrangement. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2086–2092. IEEE, 2023.
- [24] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, et al. Palm-e: an embodied multimodal language model. In *Proceedings of the 40th International Conference on Machine Learning*, pages 8469–8488, 2023.
- [25] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, pages 9118–9147. PMLR, 2022.
- [26] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone. Llm+ p: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.
- [27] S. Wang, M. Han, Z. Jiao, Z. Zhang, Y. N. Wu, S.-C. Zhu, and H. Liu. Llm⁺ 3: Large language model-based task and motion planning with motion failure reasoning. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12086–12092. IEEE, 2024.
- [28] T. Silver, S. Dan, K. Srinivas, J. B. Tenenbaum, L. Kaelbling, and M. Katz. Generalized planning in pddl domains with pretrained large language models. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 20256–20264, 2024.
- [29] H. D. Sherali and H. Wang. Global optimization of nonconvex factorable programming problems. *Mathematical Programming*, 89(3):459–478, 2001.
- [30] R. Deits and R. Tedrake. Computing large convex regions of obstacle-free space through semidefinite programming. In *Algorithmic Foundations of Robotics XI: Selected Contributions of the Eleventh International Workshop on the Algorithmic Foundations of Robotics*, pages 109–124. Springer, 2015.

- [31] A. Amice, H. Dai, P. Werner, A. Zhang, and R. Tedrake. Finding and optimizing certified, collision-free regions in configuration space for robot manipulators. In *International Workshop on the Algorithmic Foundations of Robotics*, pages 328–348. Springer, 2022.
- [32] Y. Yu, Y. Zhang, D. Zhang, X. Liang, H. Zhang, X. Zhang, Z. Yang, M. Khademi, H. Awadalla, J. Wang, et al. Chain-of-reasoning: Towards unified mathematical reasoning in large language models via a multi-paradigm perspective. *arXiv preprint arXiv:2501.11110*, 2025.
- [33] X. Xiong, S. Han, Z. Zhou, and A. Cohan. Metamath: Integrating natural language and code for enhanced mathematical reasoning in large language models. *arXiv e-prints*, pages arXiv–2409, 2024.
- [34] W. Shi, Z. Hu, Y. Bin, J. Liu, Y. Yang, S.-K. Ng, L. Bing, and R. K.-W. Lee. Math-llava: Bootstrapping mathematical reasoning for multimodal large language models. *arXiv preprint arXiv:2406.17294*, 2024.
- [35] Q. Li, L. Zhang, and V. Mak-Hau. Synthesizing mixed-integer linear programming models from natural language descriptions. *arXiv preprint arXiv:2311.15271*, 2023.
- [36] A. AhmadiTeshnizi, W. Gao, and M. Udell. Optimus: Scalable optimization modeling with (mi) lp solvers and large language models. *arXiv preprint arXiv:2402.10172*, 2024.
- [37] S. Li, J. Kulkarni, I. Menache, C. Wu, and B. Li. Towards foundation models for mixed integer linear programming. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=6yENDA7J4G>.
- [38] S. Karaman and E. Frazzoli. Sampling-based algorithms for optimal motion planning. *The international journal of robotics research*, 30(7):846–894, 2011.
- [39] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot. Batch informed trees (bit*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs. In *2015 IEEE international conference on robotics and automation (ICRA)*, pages 3067–3074. IEEE, 2015.
- [40] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE transactions on Robotics and Automation*, 12(4):566–580, 2002.
- [41] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel. Motion planning with sequential convex optimization and convex collision checking. *The International Journal of Robotics Research*, 33(9):1251–1270, 2014.
- [42] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling. Pddlstream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning. In *Proceedings of the international conference on automated planning and scheduling*, volume 30, pages 440–448, 2020.
- [43] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [44] K. Agarwal, Y. Jiang, J. Hu, B. Liu, and P. Stone. L3m+ p: Lifelong planning with large language models. *arXiv preprint arXiv:2508.01917*, 2025.
- [45] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International conference on robotics and automation (ICRA)*, pages 9493–9500. IEEE, 2023.
- [46] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg. Progprompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv:2209.11302*, 2022.

- [47] X. Sun, Y. Zhang, X. Tang, A. S. Bedi, and A. Bera. Trustnavgpt: Modeling uncertainty to improve trustworthiness of audio-guided llm-based robot navigation. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8794–8801, 2024. doi:10.1109/IROS58592.2024.10801932.
- [48] X. Sun, H. Meng, S. Chakraborty, A. S. Bedi, and A. Bera. Beyond text: Utilizing vocal cues to improve decision making in llms for robot navigation tasks. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=ojWtq4n7Ag>.
- [49] X. Sun, Z. Pan, X. Gao, K. Wu, and A. Bera. Text-guided generation of efficient personalized inspection plans. *arXiv preprint arXiv:2506.02917*, 2025.
- [50] X. Sun, S. Jia, Z. Pan, K. Wu, and A. Bera. Handle-based mesh deformation guided by vision language model. *IEEE Transactions on Visualization and Computer Graphics*, 32(7):7161–7174, 2026. doi:10.1109/TVCG.2026.3695016.
- [51] B. Li, K. Mellou, B. Zhang, J. Pathuri, and I. Menache. Large language models for supply chain optimization. *arXiv preprint arXiv:2307.03875*, 2023.
- [52] H. Chen, G. E. Constante-Flores, and C. Li. Diagnosing infeasible optimization problems using large language models. *INFOR: Information Systems and Operational Research*, 62(4): 573–587, 2024.
- [53] F. Liu, X. Tong, M. Yuan, X. Lin, F. Luo, Z. Wang, Z. Lu, and Q. Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. *arXiv preprint arXiv:2401.02051*, 2024.
- [54] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [55] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [56] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- [57] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [58] Y. Chen, J. Xia, S. Shao, D. Ge, and Y. Ye. Solver-informed rl: Grounding large language models for authentic optimization modeling. *arXiv preprint arXiv:2505.11792*, 2025.
- [59] J. Feng, S. Huang, X. Qu, G. Zhang, Y. Qin, B. Zhong, C. Jiang, J. Chi, and W. Zhong. Retool: Reinforcement learning for strategic tool use in llms. *arXiv preprint arXiv:2504.11536*, 2025.
- [60] C. Li, M. Xue, Z. Zhang, J. Yang, B. Zhang, B. Yu, B. Hui, J. Lin, X. Wang, and D. Liu. Start: Self-taught reasoner with tools. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 13523–13564, 2025.
- [61] S. Diamond and S. Boyd. Cvxpy: A python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.
- [62] J. P. Pedroso. Optimization with gurobi and python. *INESC Porto and Universidade do Porto,, Porto, Portugal*, 1, 2011.

- [63] H. Zhang, X. Gu, J. Li, C. Ma, S. Bai, C. Zhang, B. Zhang, Z. Zhou, D. He, and Y. Tang. Thinking with videos: Multimodal tool-augmented reinforcement learning for long video reasoning. *arXiv preprint arXiv:2508.04416*, 2025.
- [64] K. Hang, J. A. Stork, N. S. Pollard, and D. Kragic. A framework for optimal grasp contact planning. *IEEE Robotics and Automation Letters*, 2(2):704–711, 2017.
- [65] I. A. Sucan, M. Moll, and L. E. Kavraki. The open motion planning library. *IEEE Robotics & Automation Magazine*, 19(4):72–82, 2012.
- [66] H. Dai, A. Majumdar, and R. Tedrake. Synthesis and optimization of force closure grasps via sequential semidefinite programming. In *Robotics Research: Volume 1*, pages 285–305. Springer, 2017.
- [67] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
- [68] Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, T. Fan, G. Liu, L. Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *Advances in Neural Information Processing Systems*, 38:113222–113244, 2026.
- [69] Z. Liu, C. Chen, W. Li, P. Qi, T. Pang, C. Du, W. S. Lee, and M. Lin. Understanding rl-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- [70] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [71] C. Zhang, W. Ouyang, H. Yuan, L. Gong, Y. Sun, Z. Guo, Z. Dong, and J. Yan. Towards imitation learning to branch for mip: A hybrid reinforcement learning based sample augmentation approach. In *The Twelfth International Conference on Learning Representations*, 2024.
- [72] A. G. Labassi, D. Chételat, and A. Lodi. Learning to compare nodes in branch and bound with graph neural networks. *Advances in neural information processing systems*, 35:32000–32010, 2022.
- [73] Z. Wang, X. Li, J. Wang, Y. Kuang, M. Yuan, J. Zeng, Y. Zhang, and F. Wu. Learning cut selection for mixed-integer linear programming via hierarchical sequence model. *arXiv preprint arXiv:2302.00244*, 2023.
- [74] A. Prouvost, J. Dumouchelle, L. Scavuzzo, M. Gasse, D. Chételat, and A. Lodi. Ecole: A gym-like library for machine learning in combinatorial optimization solvers. *arXiv preprint arXiv:2011.06069*, 2020.
- [75] Z. Ren, Z. Shao, J. Song, H. Xin, H. Wang, W. Zhao, L. Zhang, Z. Fu, Q. Zhu, D. Yang, et al. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.
- [76] H. Ying, Z. Wu, Y. Geng, J. Wang, D. Lin, and K. Chen. Lean workbook: A large-scale lean problem set formalized from natural language math problems. *Advances in Neural Information Processing Systems*, 37:105848–105863, 2024.
- [77] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625:476–482, 2024. doi:10.1038/s41586-023-06747-5. URL <https://www.nature.com/articles/s41586-023-06747-5>.
- [78] Y. Chervonyi, T. H. Trinh, M. Olšák, X. Yang, H. H. Nguyen, M. Menegali, J. Jung, J. Kim, V. Verma, Q. V. Le, et al. Gold-medalist performance in solving olympiad geometry with alpheageometry2. *Journal of Machine Learning Research*, 26(241):1–39, 2025.

Appendix Contents

A Task Description	16
B LLM³P Bench Data Generation Pipeline	16
B.1 Single-Modal Data Generation	16
B.2 Multi-Modal Composition	18
C More Quantitative Results	18
C.1 Effect of In-Context Examples	18
C.2 Effect of GRPO Training Steps	19
C.3 Detailed Per-Modality Performance	20
C.4 Structured Held-Out Generalization	21
C.5 Held-Out Difficulty Generalization	22
C.6 Failure Mode Analysis	24
D More Qualitative Results	24
D.1 2D Visualization	24
D.2 3D Visualization	24
E Additional Real-World Robot Results	25
F LLM-Callable API	32
F.1 Initialization APIs	32
F.2 Constraint APIs	33
G MIP Formulation	34
G.1 UAV	34
G.2 Extension to Our Novel Spatial Constraints	35
G.3 IK	36
G.4 Grasp	36
G.5 Finger Selection	38
G.6 Footstep	38
G.7 Multi-modal Tasks	40
H Method Implementation Detail	40
H.1 Fine-tuning Detail	40
H.2 Baselines Setup Detail	41
I Additional Related Work	43
J LLM³P Bench Data Example	44
K LLM Prompt	54

A Task Description

We provide detailed descriptions of our five single-modal M³P tasks:

- **Footstep Planning** [21]: The goal is for a bipedal robot to generate a sequence of footsteps, subject to reachability and obstacle-avoidance constraints, such that its center of mass reaches a desired 2D target position.
- **Finger Selection** [64]: The goal is for a robotic gripper to select fingertip contact points that maximize the 2D grasp quality of an object.
- **Inverse Kinematics** [17]: The goal is for an articulated robotic arm to reach a target configuration while avoiding a set of obstacles.
- **Collision-Free UAV Trajectory Planning (UAV)** [18]: The goal is for a UAV to reach a target position without colliding with obstacles, while satisfying additional user-defined constraints. We support various constraint types, such as “circling around an obstacle” and “flying from the left of an obstacle.”
- **Grasp** [15]: A combination of finger selection and IK, where the gripper must select contact points and compute the corresponding arm configuration to reach them.

We further provide detailed descriptions of our multi-modal M³P tasks:

- **Footstep+IK**: A bipedal robot equipped with a 2D articulated arm must reach a distant target. It first walks closer to the target via footstep planning, then uses IK to reach it.
- **UAV+IK**: Similar to Footstep+IK, but with a UAV equipped with a 2D articulated arm, which must reach a distant target while avoiding obstacles.
- **Footstep+UAV**: A bipedal robot and a UAV must meet at a known or unknown location while avoiding obstacles.
- **Multi-UAV**: Multiple UAVs must meet at a known or unknown location while avoiding obstacles.
- **Multi-Footstep**: Multiple bipedal robots must meet at a known or unknown location while avoiding obstacles.
- **Multi-IK**: Multiple robotic arms must reach a shared target configuration or location while avoiding obstacles.
- **Footstep+IK+Grasp**: A bipedal robot must grasp a distant object. It first walks close to the object, then selects grasp points, and finally reaches the object using IK.
- **UAV+IK+Grasp**: Similar to Footstep+IK+Grasp, but with a UAV equipped with a gripper.

B LLMM³P Bench Data Generation Pipeline

As illustrated in Figure 6, we construct an *agentic data generation pipeline* that automatically produces paired natural-language missions and solver-grounded executable programs for M³P tasks. The key challenge is that expert code based on previously published papers [15, 17] is only available for individual motion-planning modalities, such as UAV navigation, inverse kinematics, finger selection, and footstep planning. There is no hand-designed expert solver that directly generates complete programs for arbitrary multi-modal compositions. Therefore, our dataset is generated by combining validated single-modality experts with LLM-based task synthesis, structured grounding, and cross-modal composition.

B.1 Single-Modal Data Generation

For each selected modality $m \in \mathcal{M}$, where $\mathcal{M} = \{\text{UAV, IK, footstep, finger selection, grasp}\}$ denotes the supported single-modal task types, a generation LLM agent $\mathcal{A}_{\text{gen}}$ first samples a task instance

$$\mathcal{E} = (\mathcal{W}, \mathcal{O}, s, g, \mathcal{G}_{\text{mod}}).$$

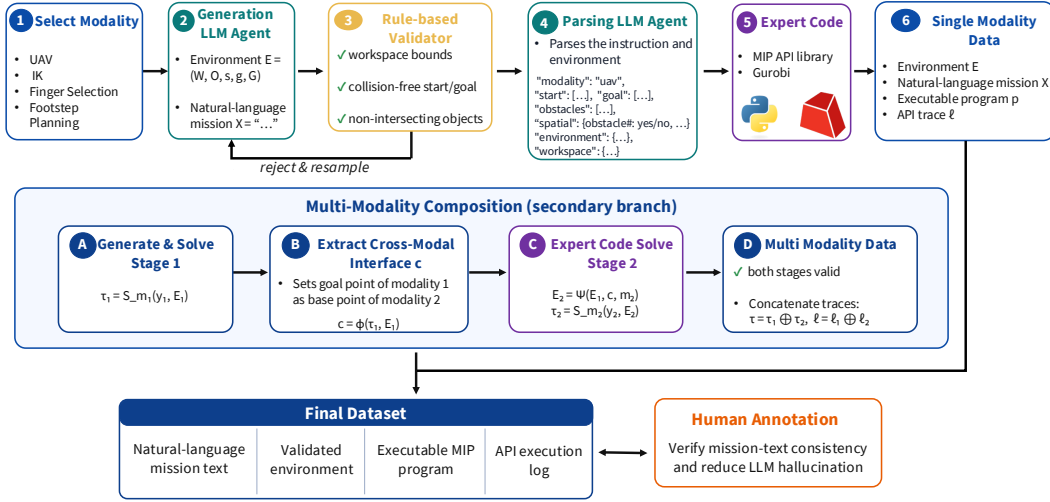


Figure 6: LLM³P Bench dataset preparation pipeline. An agentic generation pipeline constructs single-modality M³P data by sampling task environments and natural-language missions, validating geometric consistency, parsing instructions into structured representations, and solving them with modality-specific MIP expert code. Multi-modality tasks are generated through a secondary composition branch that couples feasible single-modality solutions via cross-modal interface constraints and concatenates their executable traces. Human annotation further verifies mission–data consistency before inclusion in the final dataset.

Here, $\mathcal{W}$ denotes workspace bounds, $\mathcal{O}$ is the obstacle map, s and g are start and goal states, and $\mathcal{G}_{\text{mod}}$ denotes modality-specific geometry, such as robot kinematics, UAV navigation parameters, footstep terrain regions, finger-selection candidates, or grasp geometry. The same agent also generates a natural-language mission instruction x that describes the task constraints, including spatial relations, footstep planning, or finger selection, and goal location. To encourage diverse instruction wording and environment variation, $\mathcal{A}_{\text{gen}}$ uses a relatively high sampling temperature, e.g., temperature = 0.7.

A deterministic rule-based validation module $\mathcal{V}$ then checks environment-internal geometric and semantic consistency, $\mathcal{V}(\mathcal{E}) = \mathbf{1}[\text{valid}]$. Specifically, it verifies that all entities lie within the workspace bounds, start and goal states are distinct and collision-free, and objects in $\mathcal{O} \cup \mathcal{G}_{\text{mod}}$ do not intersect; cross-modal coupling is enforced separately by the interface constraint factor in the multi-modal composition stage below. Invalid samples are rejected and resampled.

Next, a parsing LLM agent $\mathcal{A}_{\text{parse}}$ (a low-temperature parser mapping a mission and environment to a typed intermediate representation) maps the instruction and environment into a typed intermediate representation, $z = \mathcal{A}_{\text{parse}}(x, \mathcal{E})$. The IR encodes modality type, start and goal references, spatial constraints, goal constraints, using a fixed schema with restricted vocabularies. This constrained representation reduces ambiguity and ensures compatibility with downstream expert code. Since this stage emphasizes reliability rather than diversity, $\mathcal{A}_{\text{parse}}$ uses a low sampling temperature, e.g., temperature = 0.1.

Finally, let $\{\mathcal{S}_m\}_{m \in \mathcal{M}}$ denote the family of modality-specific instantiations of the generic solver $\mathcal{S}$ from Section 3. A solver module $\mathcal{S}_m$ constructs and solves the corresponding MIP, returning the pair $(\tau, \mathcal{L}) = \mathcal{S}_m(y, \mathcal{E})$, where τ is the resulting trajectory and $\mathcal{L}$ is the API execution log, and y is the executable program assembled from the IR z together with the modality- m expert code template, using modality-specific expert formulations implemented through a Python library of callable MIP-discretization APIs and Gurobi. For each solved instance, we record the full executable program, the environment parameters, the resulting trajectory τ , and the executed API sequence in `run.log.txt`.

This log provides an executable trace of the solver-grounded solution and is later used to compute the API reward R_{API} during GRPO.

Within this section the symbols z , y , and $\mathcal{L}$ have fixed meanings (typed intermediate representation, executable MIP program, and API execution log, respectively) and should not be confused with same-letter symbols used locally inside the modality subsections of Appendix G. Each single-modality data point is therefore represented as the tuple $(x, \mathcal{E}, z, y, \tau, \mathcal{L})$, where z is the typed intermediate representation, y is the executable MIP program, and $\mathcal{L}$ is the API execution log.

B.2 Multi-Modal Composition

For multi-modal motion-planning tasks, direct expert code is not available. Instead, we construct compositional tasks by coupling multiple validated single-modal tasks. Consider a two-stage composition with modalities m_1 and m_2 . Let $\mathbb{T}$ denote the space of feasible trajectories, $\mathbb{E}$ the space of task environments defined above, and $\mathcal{C}$ the space of cross-modal interface constraints (e.g., shared waypoints or contact poses). We first generate and solve a valid single-modal instance, $(\tau_1, \mathcal{L}_1) = \mathcal{S}_{m_1}(y_1, \mathcal{E}_1)$. The terminal state or selected waypoint of the first solution is then used to define the start state, goal state, rendezvous point, or base pose for the second modality via $\iota = \phi(\tau_1, \mathcal{E}_1)$, where $\phi : \mathbb{T} \times \mathbb{E} \rightarrow \mathcal{C}$ is an interface-extraction map that selects a coupling waypoint or pose from the first-stage trajectory and environment, and $\iota \in \mathcal{C}$ denotes the resulting cross-modal interface constraint. We then instantiate the second modality conditioned on this interface:

$$\mathcal{E}_2 = \Psi(\mathcal{E}_1, \iota, m_2), \quad (\tau_2, \mathcal{L}_2) = \mathcal{S}_{m_2}(y_2, \mathcal{E}_2),$$

where $\Psi : \mathbb{E} \times \mathcal{C} \times \mathcal{M} \rightarrow \mathbb{E}$ instantiates a modality- m_2 task environment by extending $\mathcal{E}_1$ with the interface ι and any modality- m_2 -specific geometry. Only compositions for which both stages are feasible and the interface constraint is satisfied are retained:

$$\mathbf{1}[\text{valid}] = \mathcal{V}(\mathcal{E}_1) \cdot \mathcal{V}(\mathcal{E}_2) \cdot \mathbf{1}[C_{\text{iface}}(\tau_1, \tau_2, \iota) = 1].$$

Here $C_{\text{iface}}(\tau_1, \tau_2, \iota) \in \{0, 1\}$ is a predicate indicating whether the trajectories τ_1 and τ_2 satisfy the interface constraint ι . $\mathcal{V}$ checks only environment-internal validity (workspace bounds, obstacle non-intersection, distinct start/goal), while cross-modal coupling is enforced by the separate factor $\mathbf{1}[C_{\text{iface}}(\tau_1, \tau_2, \iota) = 1]$.

For example, in a UAV-IK task, the UAV expert first solves a navigation problem and produces a terminal position. This terminal position is then used as the base pose for the IK problem, which is solved by the IK expert code. The final multi-modal solution is obtained by concatenating the two executable traces, $\tau = \tau_1 \oplus \tau_2$ and $\mathcal{L} = \mathcal{L}_1 \oplus \mathcal{L}_2$ (where $\oplus$ denotes sequential concatenation of trajectories, resp. of API-call logs), where the combined `run_log.txt` contains the API calls and trajectory-generation steps for both modalities. This procedure guarantees feasibility of the constructed multi-modal instance, which is sufficient for fine-tuning and solver-grounded evaluation. The concatenated trajectory τ need not be globally optimal for the full composed task, since the model is not rewarded for matching the reference trajectory exactly, but for generating an executable program that satisfies the mission constraints. To balance diversity and solve efficiency, we sample moderate-sized environments with 1–3 agents and 2–8 obstacles.

Lastly, human annotators verify that every data sample matches the textual constraints in the mission instruction, ensuring data validity and reducing the effects of possible LLM hallucinations. The resulting dataset contains natural-language missions, validated environments, executable MIP programs, and API execution logs for both single- and multi-modal M³P tasks. We showcase data samples in LLMM³P Bench in Appendix J.

C More Quantitative Results

C.1 Effect of In-Context Examples

Figure 7 compares the performance of different LLMs under varying numbers of in-context demonstrations. We compare 1/2/4 shots of in-context examples (single-modality) and observe that in-

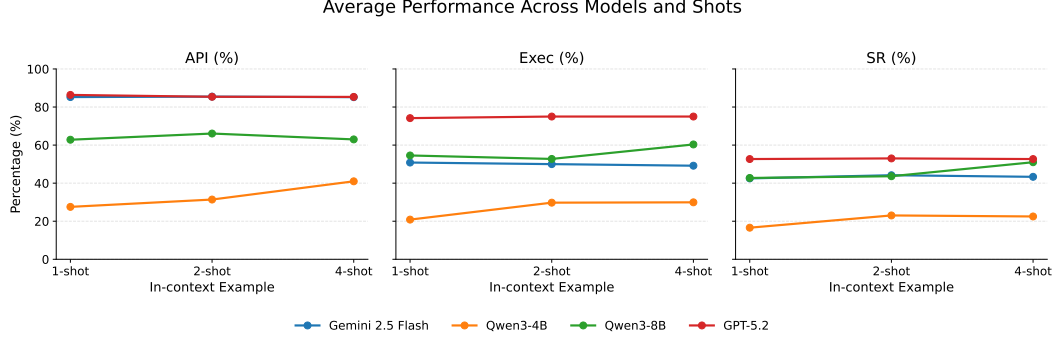


Figure 7: Effect of increasing the number of in-context examples for direct prompting. We evaluate 1/2/4-shot prompting using only single-modal demonstrations and test the resulting prompts on both single- and multi-modality M³P tasks. Each panel reports one metric (API %, Exec %, or SR %) on the y-axis; the x-axis sweeps the number of in-context shots (1/2/4), and colors denote different LLMs. Across different LLMs, increasing the number of demonstrations does not consistently improve API accuracy, execution success, or task correctness, indicating that M³P reasoning cannot be reliably induced through prompting alone.

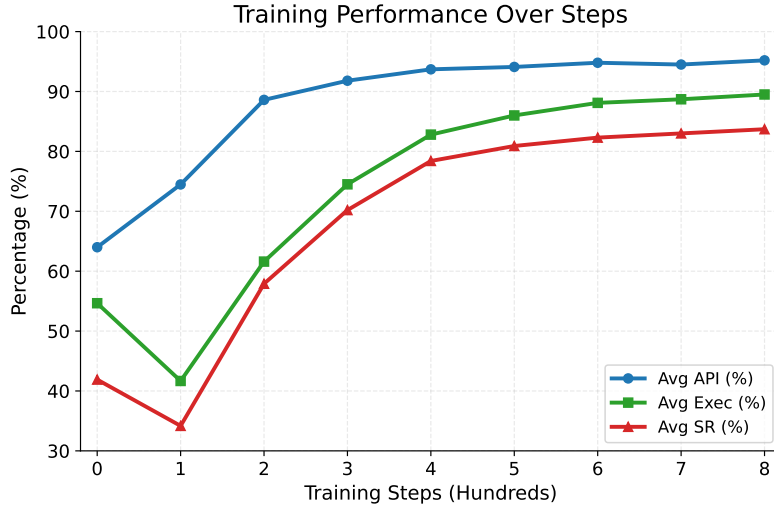


Figure 8: Performance improvement across different GRPO training steps. API accuracy, execution success, and task correctness improve rapidly during the early stages of training and gradually saturate after approximately 500 steps, with only minor fluctuations afterward.

context learning does not scale for our tasks. While larger proprietary models such as GPT-5.2 and Gemini-2.5-Flash achieve relatively strong performance, additional examples provide only marginal or inconsistent improvements. Smaller open-source models exhibit larger variance and limited gains from prompting alone. Overall, the results suggest that M³P tasks require structured solver-grounded reasoning beyond simple in-context learning, motivating the need for dedicated fine-tuning.

C.2 Effect of GRPO Training Steps

Figure 8 illustrates the effect of GRPO optimization on M³P performance. The model shows substantial gains in API accuracy, execution success, and task correctness during the first several hundred training steps, demonstrating that GRPO effectively improves solver-grounded reasoning abilities. After around 500 training steps, the performance begins to stabilize and approaches saturation, indicating convergence toward a robust policy for structured robotic planning.

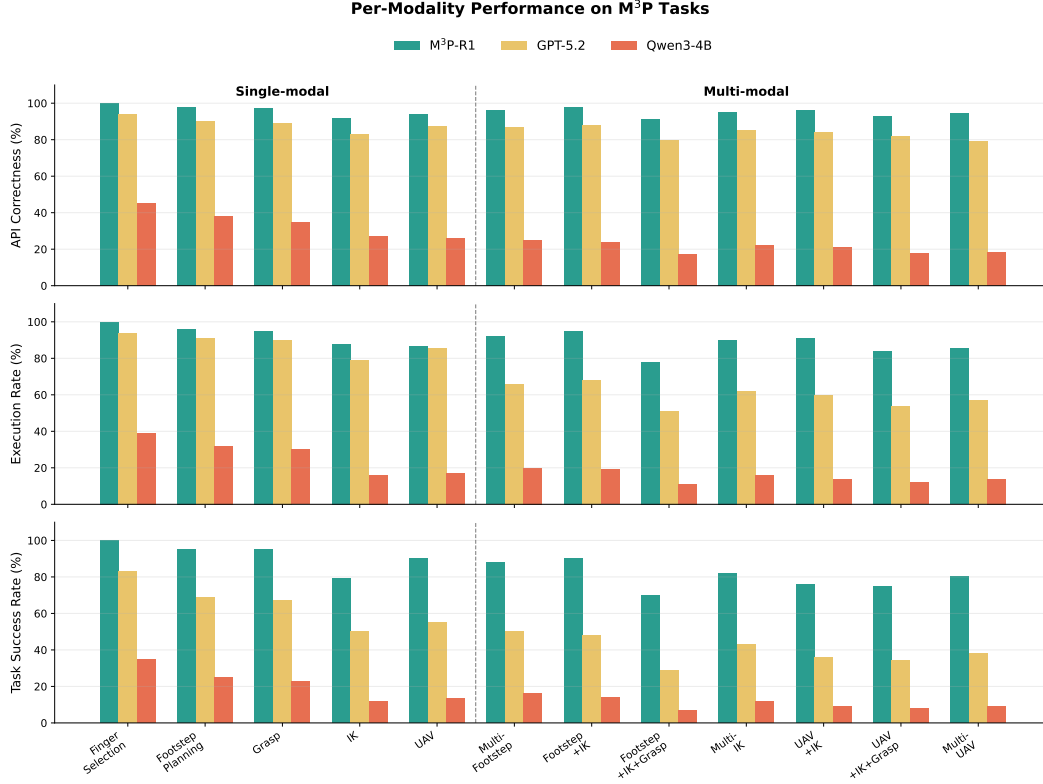


Figure 9: Per-modality performance on single- and multi-modality M³P tasks. We compare M³P-R1 with GPT-5.2 and Qwen3-4B across API correctness, execution rate, and task success rate. M³P-R1 maintains strong performance across both single-modal and multi-modal planning domains, with the largest gains appearing on complex multi-modal tasks.

C.3 Detailed Per-Modality Performance

Figure 9 provides a detailed performance breakdown across different single- and multi-modality M³P tasks. Overall, M³P-R1 achieves strong and consistent performance on single-modality planning problems, with near-perfect results on finger selection, footstep planning, and grasp. More challenging continuous optimization tasks, such as IK and UAV, show lower execution and task success rates due to the increased difficulty of geometric reasoning, collision avoidance, and constraint satisfaction.

Performance on multi-modality tasks remains strong but decreases compared with single-modality settings, especially for tasks requiring joint reasoning over multiple coupled decision spaces. In particular, `footstep+ik+grasp` and `uav+ik+grasp` produce the lowest execution and task success rates because the model must simultaneously coordinate locomotion, inverse kinematics, grasp feasibility, and collision-free planning within a unified MIP formulation. Nevertheless, M³P-R1 consistently outperforms GPT-5.2 and Qwen3-4B across all modalities. The gap becomes larger as task complexity increases, especially in execution success and task correctness for multi-modal planning problems. As shown in Figure 9, general-purpose LLM baselines experience substantial degradation when solving complex multi-modal tasks, whereas M³P-R1 maintains relatively stable API correctness and execution performance across diverse planning domains. These results suggest that RFT enables the model to learn transferable solver-grounded planning strategies rather than modality-specific program templates.

Table 5: Held-out composition generalization during RFT. Complex compositions (uav+ik+grasp and footstep+ik+grasp) are excluded from RFT training but evaluated during testing. We compare against GPT-5.2, the strongest non-fine-tuned LLM baseline (same model used across all held-out rows).

Setting	Ours (%)			Decrease (%)			Gain over Best LLM (%)		
	API	Exec	SR	API	Exec	SR	API	Exec	SR
Single-modal	96.8	92.4	92.1	+0.6	-0.7	+0.3	+8.1	+4.5	+27.3
Multi-modal Seen	92.4	85.4	76.9	-3.5	-5.3	-6.4	+8.8	+25.7	+37.2
Held-out Composition	88.6	71.8	60.2	-3.4	-9.2	-12.3	+7.2	+18.1	+22.8

C.4 Structured Held-Out Generalization

Note on independent runs and ID test reconciliation. Each held-out split below entails a fully independent SFT+GRPO training run with the corresponding subset of tasks excluded; the *Multi-modal Seen* rows therefore vary across Table 5, 6, and 7 because each row reflects a separate training run. For the same reason, the in-distribution (ID) test single-/multi-modal SR reported in this section (91.8%/80.2%; Table 10) differs slightly from the headline 91.7%/80.9% in Table 1: the headline number is averaged over three independent training runs, while the held-out tables report a single training run per row. Differences smaller than the ID-test run-to-run std (approximately 1.5 pp) should therefore not be over-interpreted.

To assess whether M³P-R1 learns reusable solver-grounded planning strategies rather than memorizing fixed program templates, we evaluate three structured held-out settings. In the first setting, the complex compositions uav+ik+grasp and footstep+ik+grasp are excluded during RFT training. In the second and third settings, we remove all grasp-related or UAV-related tasks from fine-tuning, respectively, and evaluate them only at test time. These splits are substantially harder than random test splits because they require the model to compose MIP primitives and cross-modal interfaces in unseen configurations. Note that this setting evaluates ID generalization to held-out structural splits, as opposed to the out-of-distribution (OOD) difficulty-axis generalization in Appendix C.5.

Metrics. We evaluate generated programs using both LLM-level and robotics-level metrics. API measures whether the generated code uses valid discretization APIs with correct argument structure. Exec measures whether the generated program is syntactically valid and successfully invokes the optimization solver. SR measures full task success, requiring the executed solution to satisfy the complete task specification. For robotics-level evaluation, Traj. SR measures whether the returned trajectory satisfies start and goal constraints, continuity, region membership, and robot-specific feasibility constraints. Sem. SR measures whether the trajectory satisfies the language-specified spatial relations and task semantics like robot meet. Collision-free rate measures the percentage of executed trajectories that avoid all obstacles. Path length is normalized by the ground-truth expert MIP solution, with 1.0 corresponding to expert-level path efficiency.

As shown in Table 5, 6, and 7, performance decreases compared with the fully fine-tuned setting, especially in execution success and task success rate. This degradation is expected because the model must solve unseen modality compositions or reason about held-out robotic primitives without direct fine-tuning examples. Nevertheless, M³P-R1 remains consistently stronger than **GPT-5.2**, the strongest non-fine-tuned LLM baseline in Table 1 (the same model is used across all held-out rows). On held-out composition tasks, M³P-R1 improves over GPT-5.2 by +7.2% API accuracy, +18.1% execution success, and +22.8% task success rate. Even in the more challenging held-out primitive settings, it still achieves gains of +2.6%/+18.8%/+16.0% on grasp-related tasks and +3.5%/+30.9%/+25.5% on UAV-related tasks. These results suggest that the model is not simply memorizing training templates. Instead, RFT helps it learn transferable robotic MIP formulation patterns, including API composition, solver invocation, and cross-modal constraint grounding.

Table 6: Held-out grasp generalization. Grasp-related tasks are excluded from both SFT and RFT training and evaluated only during testing. We compare against GPT-5.2, the strongest non-fine-tuned LLM baseline (same model used across all held-out rows).

Setting	Ours (%)			Decrease (%)			Gain over Best LLM (%)		
	API	Exec	SR	API	Exec	SR	API	Exec	SR
Single-modal Seen	96.3	92.1	91.4	+0.3	-0.5	+0.4	+7.6	+4.2	+26.6
Multi-modal Seen	89.1	82.6	73.8	-6.8	-8.1	-9.5	+5.5	+22.9	+34.1
Held-out Grasp Tasks	84.2	63.5	50.7	-9.5	-22.2	-29.3	+2.6	+18.8	+16.0

Table 7: Held-out UAV generalization. UAV-related tasks are excluded from both SFT and RFT training and evaluated only during testing. We compare against GPT-5.2, the strongest non-fine-tuned LLM baseline (same model used across all held-out rows).

Setting	Ours (%)			Decrease (%)			Gain over Best LLM (%)		
	API	Exec	SR	API	Exec	SR	API	Exec	SR
Single-modal Seen	95.4	94.2	89.8	-1.4	-0.6	-2.5	+6.7	+6.3	+25.0
Multi-modal Seen	91.8	82.4	74.1	-3.2	-6.4	-8.4	+8.2	+22.7	+34.4
Held-out UAV Tasks	84.1	62.6	55.2	-10.3	-24.1	-25.2	+3.5	+30.9	+25.5

Table 8 further evaluates whether the generated programs remain physically feasible and semantically aligned under structured held-out modality shifts. Among the three settings, Held-out Composition is the most challenging, achieving 82.0% trajectory success, 75.3% semantic success, and 80.0% collision-free rate, while also producing the longest normalized paths (1.68). This degradation is expected because the model must compose unseen cross-modal interfaces and MIP primitives without direct RFT supervision. Held-out Grasp Tasks achieve 78.2% trajectory success and 76.9% collision-free rate, indicating that the model can still construct physically valid grasp-related optimization programs despite never observing grasp tasks during fine-tuning. Held-out UAV Tasks produce the lowest trajectory success and collision-free rate, with 76.1% and 74.3%, respectively, reflecting the additional geometric and motion-planning complexity of UAV dynamics. Nevertheless, all held-out settings maintain relatively high semantic success and feasible trajectory generation, demonstrating that M³P-R1 learns transferable solver-grounded planning strategies rather than memorizing fixed modality-specific templates.

Table 8: Robotics-level metrics on held-out modality splits. Traj. SR measures physical trajectory feasibility, while Sem. SR measures satisfaction of language-specified spatial and task-level requirements. Path length is normalized by the expert MIP solution, where lower is better.

Evaluation split	Traj. SR (%)	Sem. SR (%)	Collision-free (%)	Path length ↓
Held-out Composition	82.0	75.3	80.0	1.68
Held-out Grasp Tasks	78.2	74.4	76.9	1.12
Held-out UAV Tasks	76.1	75.6	74.3	1.55

C.5 Held-Out Difficulty Generalization

We further evaluate whether M³P-R1 generalizes to task configurations that are systematically harder than those observed during fine-tuning. This study constructs controlled held-out difficulty splits along three axes: obstacle density, agent count, and spatial-relation complexity. These factors directly affect geometric planning difficulty, multi-agent coordination, constraint construction, and language-grounded task satisfaction.

We define obstacle difficulty by the number of obstacles in the environment. Simple tasks contain 0–2 obstacles, medium tasks contain 3–5 obstacles, and hard tasks contain 6–8 obstacles. Agent difficulty is defined by the number of simultaneously planned agents, ranging from one to three. Spatial difficulty is defined by the number and diversity of spatial relations specified in the language

Table 9: Held-out difficulty splits for evaluating out-of-distribution generalization. ID denotes the original in-distribution test setting. Each OOD split removes the target difficulty regime from both SFT and RFT training and evaluates only on the held-out regime.

Split	Train obstacles	Test obstacles	Train agents	Test agents	Train spatial constraints	Test spatial constraints
ID test	0–8	0–8	1–3	1–3	0–8 constraints	0–8 constraints
Obstacle-OOD	0–5	6–8	1–3	1–3	0–8 constraints	0–8 constraints
Agent-OOD	0–8	0–8	1–2	3	0–8 constraints	0–8 constraints
Spatial-OOD	0–8	0–8	1–3	1–3	≤ 1 constraint	2–8 constraints

Table 10: Held-out difficulty generalization of M³P-R1. ID denotes the original in-distribution test set. OOD splits evaluate extrapolation to harder obstacle, agent, spatial, and compositional settings. Gain is computed over GPT-5.2, the strongest non-fine-tuned LLM baseline (same model used across all rows).

Evaluation split	Single-modal Ours (%)			Multi-modal Ours (%)			Single-modal Gain (%)			Multi-modal Gain (%)		
	API	Exec	SR	API	Exec	SR	API	Exec	SR	API	Exec	SR
ID test	96.2	93.1	91.8	94.8	87.9	80.2	+7.5	+5.2	+27.0	+11.2	+28.2	+40.5
Obstacle-OOD	93.8	89.6	85.7	91.8	83.4	70.5	+5.1	+1.7	+20.9	+8.2	+23.7	+30.8
Agent-OOD	94.3	90.8	89.9	92.6	84.7	75.9	+5.6	+2.9	+25.1	+9.0	+25.0	+36.2
Spatial-OOD	94.6	91.2	87.4	93.1	85.6	76.8	+5.9	+3.3	+22.6	+9.5	+25.9	+37.1

instruction. We consider five spatial relation types: *top*, *bottom*, *left*, *right*, and *circle*. Each obstacle can be associated with at most one spatial relation.

Table 9 summarizes the held-out difficulty splits. The ID test split follows the original benchmark distribution and serves as the in-distribution reference. In Obstacle-OOD, training contains only simple and medium obstacle layouts (0–5 obstacles), while testing evaluates hard layouts with 6–8 obstacles. Agent-OOD trains on one- and two-agent tasks and evaluates three-agent coordination. Spatial-OOD trains on instructions with at most one spatial constraint and tests denser language-grounded specifications with 2–8 spatial constraints.

Table 10 reports LLM-level program-generation performance under these held-out shifts. On the ID split, M³P-R1 achieves 91.8% and 80.2% SR for single-modal and multi-modal tasks, respectively. Performance decreases gradually under held-out settings rather than collapsing. Obstacle-OOD is the most challenging split, reducing SR to 85.7% and 70.5%, reflecting the increased difficulty of dense geometric reasoning. Agent-OOD achieves 89.9% and 75.9% SR, while Spatial-OOD achieves 87.4% and 76.8% SR, demonstrating robust generalization to harder coordination and denser spatial constraints. Across all held-out splits, M³P-R1 consistently outperforms GPT-5.2 (the strongest non-fine-tuned LLM baseline; same model used across all rows) by large margins. Table 11: Robotics-level metrics on held-out difficulty splits. Traj. SR measures physical trajectory feasibility, while Sem. SR measures satisfaction of language-specified spatial and task-level requirements. Path length is normalized by the expert MIP solution, where lower is better.

Evaluation split	Traj. SR (%)	Sem. SR (%)	Collision-free (%)	Path length ↓
ID test	87.9	89.3	95.8	1.11
Obstacle-OOD	78.2	86.4	81.5	1.31
Agent-OOD	83.1	85.6	89.8	1.20
Spatial-OOD	86.1	84.7	92.3	1.07

Table 11 evaluates whether the executed solutions remain physically meaningful under the same shifts. On the ID split, M³P-R1 achieves 87.9% trajectory success, 89.3% semantic success, and 95.8% collision-free rate. Obstacle-OOD produces the largest degradation, reducing trajectory success to 78.2% and increasing normalized path length to 1.31, indicating that dense obstacle layouts mainly challenge geometric feasibility and path efficiency. Agent-OOD primarily stresses multi-agent coordination, while Spatial-OOD mainly affects semantic alignment. Nevertheless, the generated programs remain largely feasible, safe, and semantically aligned across all held-out settings.

Taken together, Table 10 and 11 show that M³P-R1 generalizes beyond the difficulty regimes observed during training. Although held-out obstacle, agent, and spatial shifts reduce performance,

the model maintains high program validity, execution success, trajectory feasibility, and collision-free performance. These results suggest that solver-grounded fine-tuning enables the model to learn reusable MIP-programming abstractions that transfer to harder planning configurations.

C.6 Failure Mode Analysis

To better understand the remaining limitations of M³P-R1, we categorize unsuccessful outputs into four mutually exclusive failure modes, as shown in Figure 10. The largest category is execution failure (42%), where the generated program is not runnable due to syntax errors, undefined variables, malformed solver calls, or incompatible API signatures. The second major category is correct API selection with insufficient discretization resolution (30%), where the model selects the appropriate planning primitive but provides suboptimal arguments, such as too few IRIS regions or an overly coarse convex decomposition, making the resulting MIP difficult or impossible to solve. Another 18% of failures come from incorrect API selection, for example choosing a rotation-discretization constraint for a UAV navigation task, which leads to an invalid or unsolvable optimization formulation. Finally, only 10% of failures are semantic failures after successful execution and solving, such as satisfying collision constraints but violating the intended spatial style, e.g., flying around the wrong side of an obstacle or mismatching the specified start/end relation. These four categories are scoped to the simulation evaluation; for the orthogonal real-world failure taxonomy (sim-to-real mesh mismatch, Tello drift, wrist singularities), see the per-failure breakdown in Appendix E.

This analysis suggests that the dominant remaining challenges are formulation robustness and solver compatibility rather than high-level task understanding. In particular, 90% of failures occur before a semantically valid solved trajectory can be produced, while only a small fraction correspond to incorrect task intent after successful solving. This indicates that once M³P-R1 constructs an executable and solvable MIP program, it usually preserves the intended task constraints. These findings also point to concrete directions for improving robustness, including stricter API signature checking, automatic repair of non-runnable code, and adaptive discretization or region-refinement strategies for difficult planning instances.

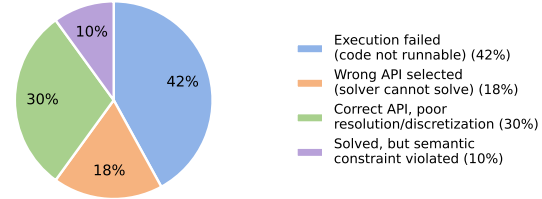


Figure 10: Failure mode analysis of unsuccessful M³P-R1 outputs across all single- and multi-modal tasks. Most failures occur before obtaining a valid task solution, either because the generated code is not executable or because the resulting optimization problem is not solver-compatible. Semantic failures after successful execution and solving are comparatively rare.

D More Qualitative Results

D.1 2D Visualization

Figure 11, 12, 13, 14, 15, 16, and 17 show 2D qualitative results for all multi-modal motion planning tasks. These 2D visualizations provide a clear view of the geometric structure of each solution, including obstacle avoidance, spatial coordination, IK linkages, grasp configurations, and trajectory feasibility.

D.2 3D Visualization

Here is our qualitative results in MuJoCo physical simulator. We deploy representative platforms for each modality, including Unitree G1 for footstep planning, Franka FR3 for inverse kinematics, Robotiq 2F-85 for grasping, Skydio X2 for UAV navigation, and Google Robot for UAV+Grasp due to the lack of real UAV-arm platforms. For each figure, the mission instruction prompt is shown below in the white space.

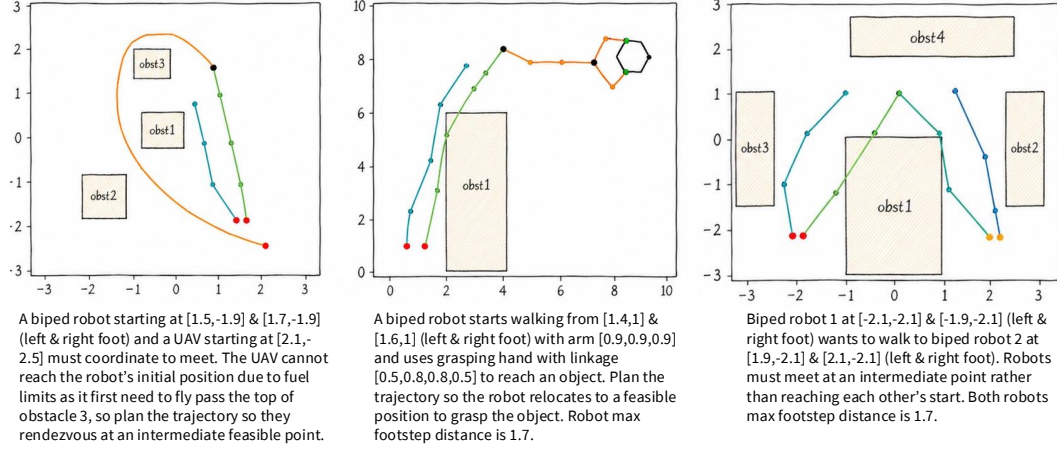


Figure 11: 2D qualitative results for the Multi-UAV task. Sub-panels show the IRIS-region-decomposed workspace with two UAV trajectories (orange: UAV1; blue: UAV2), obstacles as gray rectangles, and the shared rendezvous point as a green star. Spatial-style constraints (e.g., *circle the central obstacle*) are visualized via colored region overlays.

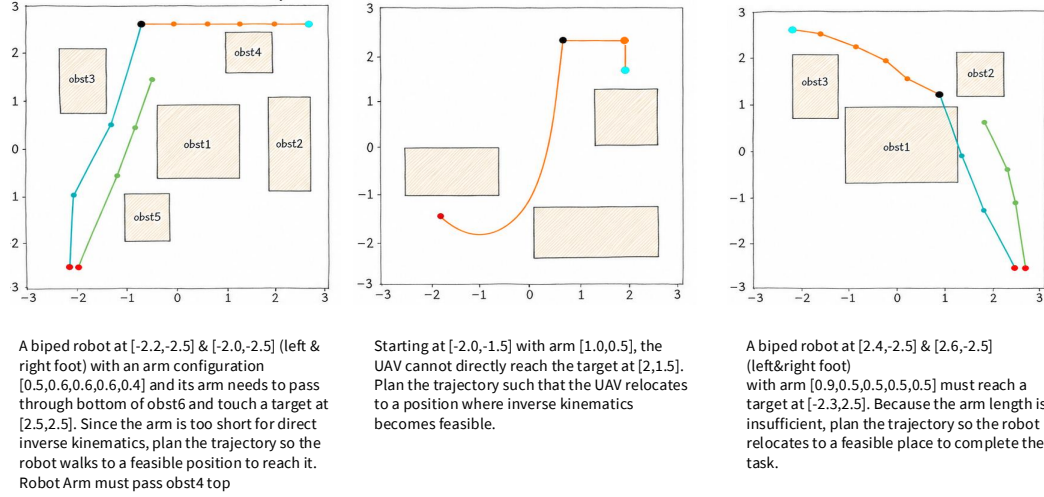


Figure 12: 2D qualitative results for the UAV+IK task. Sub-panels show the UAV trajectory (orange) reaching a handoff position (green star), followed by the articulated arm (gray links) executing inverse kinematics to the target (red marker). Obstacles are shown as gray rectangles; IRIS regions overlay the workspace.

Figure 18 and 19 show 3D qualitative results in MuJoCo simulation, demonstrating that the generated plans are feasible, collision-free, and consistent with the task constraints in full 3D environments.

E Additional Real-World Robot Results

We further evaluate M³P-R1 in real-world robotic environments using Franka Panda arms and DJI Tello UAVs. For each scene, we reconstruct the laboratory setup with the RealityScan app on an iPhone, simplify the scan into object poses, workspace bounds, and obstacle regions, and run M³P planning on a laptop. For robot-arm tasks, M³P-R1 predicts the final end-effector pose used as the target for inverse kinematics; we then use the Open Motion Planning Library (OMPL) [65] to compute a collision-aware joint-space trajectory from the initial configuration to the selected IK

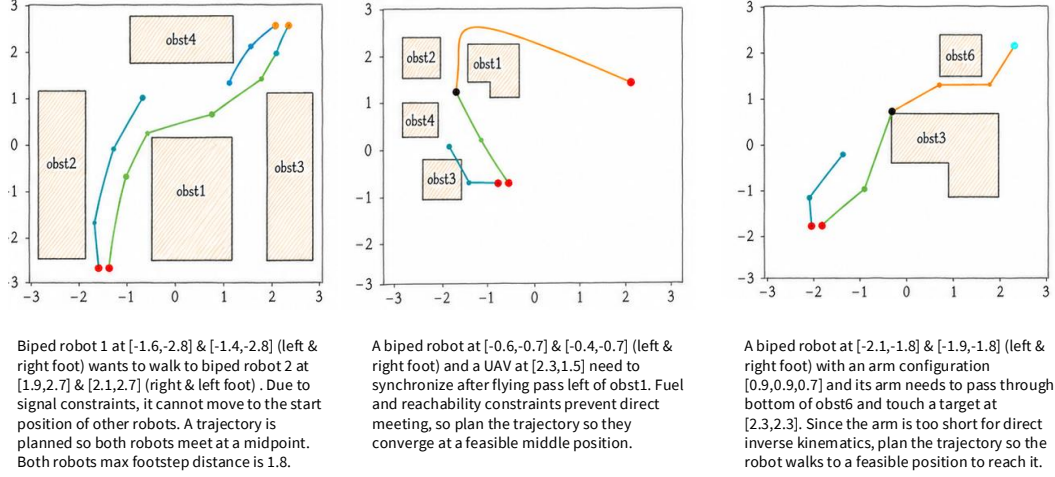


Figure 13: 2D qualitative results for the Footstep+IK task. Sub-panels show the bipedal footstep trajectory (alternating left/right contacts as red and blue squares) walking to a stance position, followed by the articulated arm reaching the target (red marker). The shared base location couples the two stages.

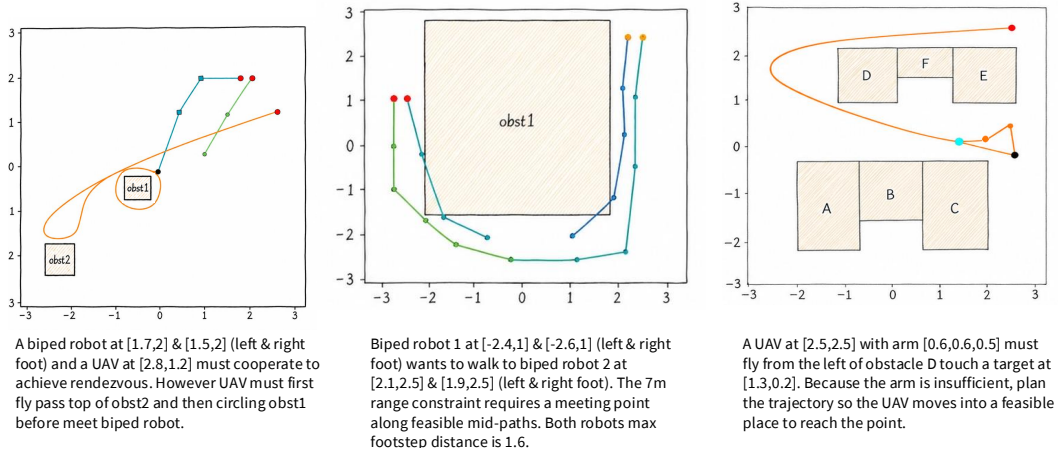


Figure 14: 2D qualitative results for the Multi-Footstep task. Sub-panels show two bipedal robots planned jointly (left robot: red/blue contacts; right robot: orange/cyan contacts) meeting at a shared rendezvous point (green star) while avoiding obstacles.

solution. The resulting robot-arm trajectories and UAV waypoints are executed using franky and DJTelloPy, respectively. Because our footstep formulation does not model full-body dynamics and lack of access to a physical bipedal platform and a closed-loop tactile gripper for safe contact-rich experiments, the real-world validation focuses on UAV planning, robot-arm inverse kinematics, and their composition. Fourth, real-world validation covers only IK, UAV, Multi-UAV, and UAV+IK tasks. Extending real-world coverage to these modalities is an important direction for future work.

Table 12 summarizes the real-world results. For each modality, we construct 10 scenarios, including 5 with object-relative spatial constraints and 5 without spatial constraints. We report collision-free execution, spatial-constraint alignment, and overall task success. Most failures are caused by sim-to-real mismatch between the coarse reconstructed mesh and the physical scene, which can lead to missing feasible paths, unexpected real-world collisions, or robot-arm configurations that are feasible in simulation but fall near kinematic singularities during execution. For Tello UAV experiments, rendezvous is evaluated within a tolerance radius of 0.5 m rather than requiring physical co-location;

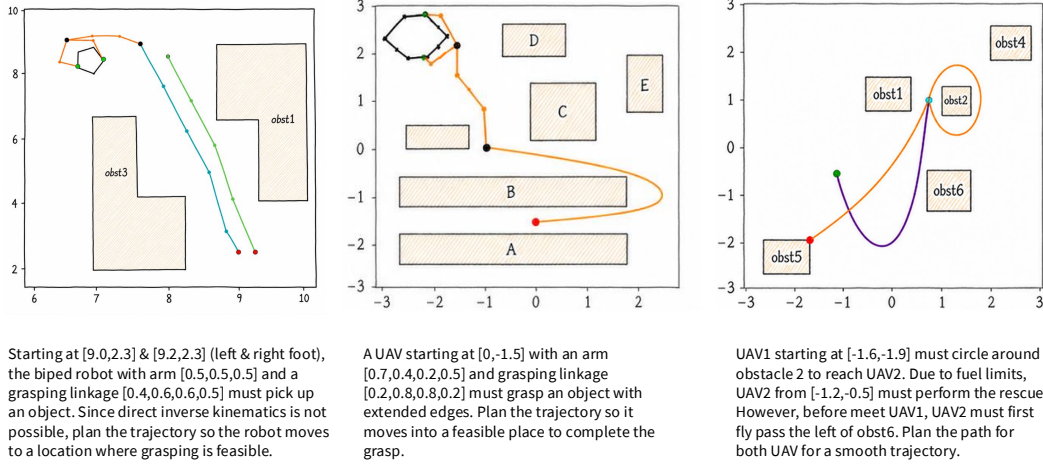


Figure 15: 2D qualitative results for the Multi-ik task. Sub-panels show two articulated arms (gray and brown links) reaching a shared target configuration (red marker) while keeping their links collision-free; per-arm pivots are marked with colored dots.

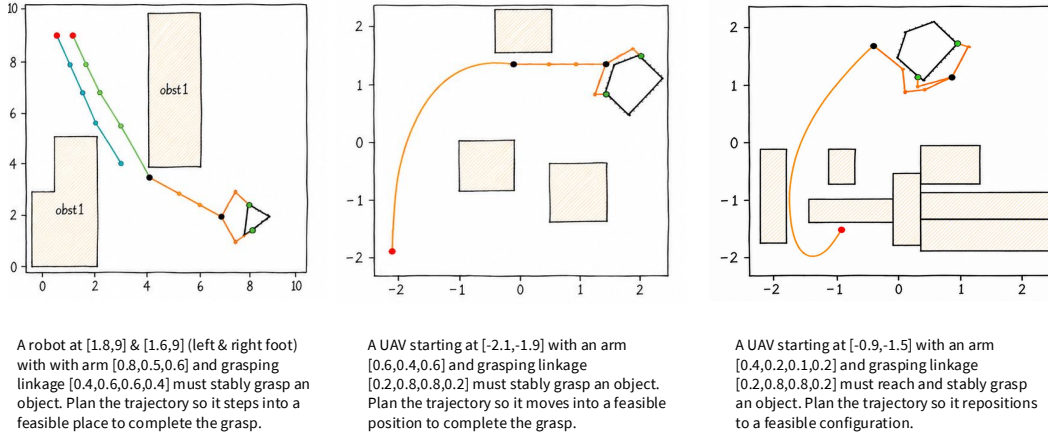


Figure 16: 2D qualitative results for the Footstep+ik+Grasp task. Sub-panels show the bipedal footstep trajectory (red/blue contacts) walking to a stance pose, the articulated arm executing IK (gray links), and the selected grasp contact points on the target object (blue dots). All three stages are coupled through shared base/target positions.

this threshold is chosen to match the onboard positioning accuracy of the Tello platform (drift on the order of 0.3–0.5 m during indoor station-keeping) and to maintain safe inter-UAV separation in close-proximity flight.

Per-failure breakdown. We summarize the five failed trials below. (1) *IK-spatial (cube near pitcher)*. The reconstructed pitcher mesh under-approximated the handle volume; the executed joint trajectory clipped the handle, triggering the real-arm collision monitor. (2) *IK-non-spatial (beam reach)*. The selected IK solution lay near a wrist singularity; OMPL planning succeeded in joint space, but the real arm aborted with a velocity-limit fault during the last segment. (3) *Multi-UAV-non-spatial*. Tello 2 drifted ~ 0.7 m from the planned waypoint before rendezvous; this exceeded the 0.5 m tolerance and triggered an emergency hover. (4) *UAV+IK-spatial*. The UAV reached the handoff position, but the meshed table edge was offset by ~ 4 cm, causing the IK end-effector pose to graze the table on descent. (5) *UAV+IK-non-spatial*. The Tello lost its visual lock briefly during the descent toward the handoff point and ended ~ 0.6 m off the planned location, exceeding the

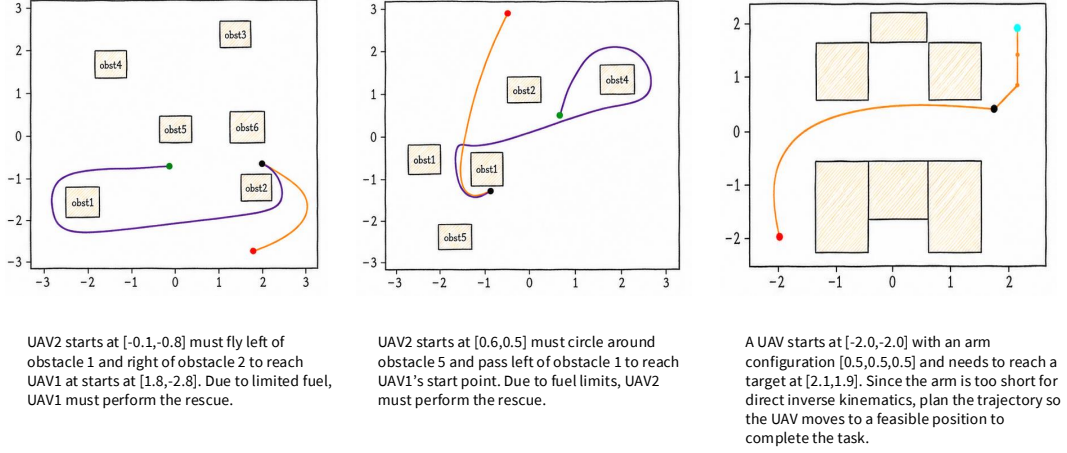


Figure 17: 2D qualitative results for the UAV+IK+Grasp task. Sub-panels show the UAV trajectory (orange) reaching a handoff position (green star), the articulated arm performing IK (gray links) attached at the UAV terminal, and the selected grasp contact points on the target object (blue dots). All stages are coupled through shared UAV terminal and IK base/target positions.

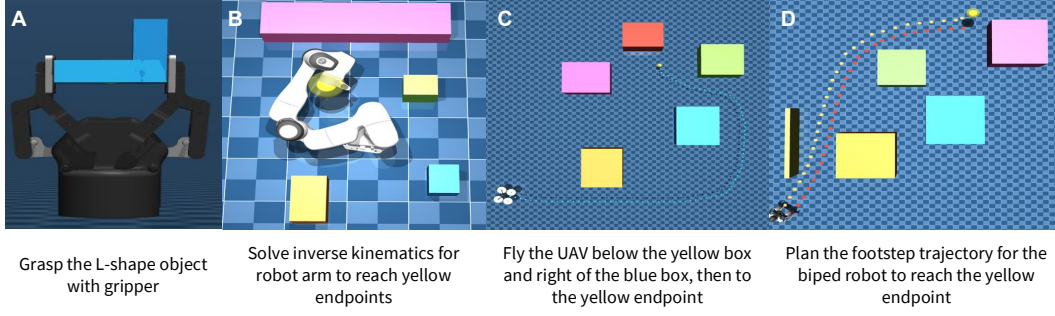
Table 12: Real-world execution results across 40 hardware trials. Each modality contains 10 scenarios, including 5 spatially constrained and 5 non-spatial tasks. Spatial alignment is evaluated only on the five spatially constrained scenarios. *Success* is defined as collision-free execution that additionally satisfies any specified spatial constraint and, for composition tasks, achieves the inter-platform rendezvous within the 0.5 m Tello tolerance.

Modality	# Scenarios	Spatial / Non-spatial	Collision-free	Spatial alignment	Success
IK	10	5 / 5	8 / 10	4 / 5	8 / 10
UAV	10	5 / 5	10 / 10	5 / 5	10 / 10
Multi-UAV	10	5 / 5	9 / 10	5 / 5	9 / 10
UAV+IK	10	5 / 5	8 / 10	4 / 5	8 / 10

tolerance. Three of the five failures are therefore directly attributable to coarse mesh reconstruction (sim-to-real mismatch), and two to onboard sensing limitations of the Tello platform.

Figure 20 and 21 show Franka Panda reaching tasks with and without object-relative spatial constraints. The constrained cases require the robot to approach the target from a specified side of a nearby object, while the unconstrained cases only require reaching the target. Figure 22 shows multi-UAV and UAV+IK composition tasks, demonstrating that M^3P-R1 can coordinate shared meeting points and spatial constraints across heterogeneous platforms.

Single Modality Motion Planning Tasks



Multi-Modality Motion Planning Tasks

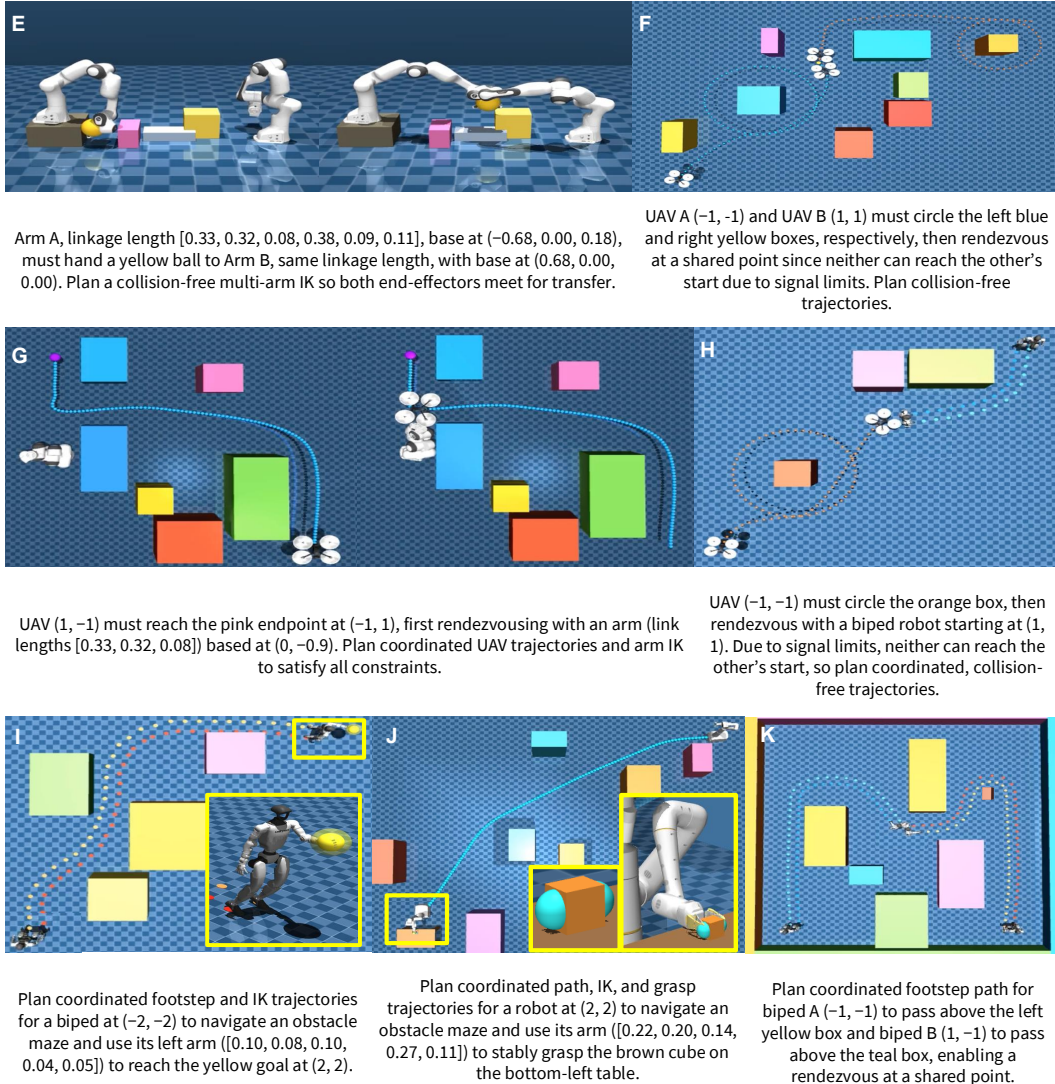
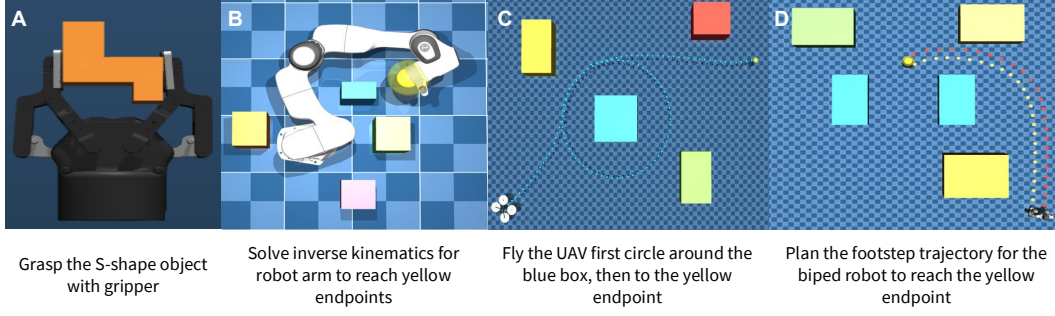


Figure 18: Qualitative results in MuJoCo simulation across single- and multi-modal M³P tasks. (A–D) Single-modal tasks: grasping, inverse kinematics (IK), UAV trajectory planning, and biped footstep planning. (E–K) Multi-modal tasks, including multi-arm coordination, multi-UAV rendezvous, UAV–arm cooperation, UAV–biped coordination, footstep+IK, full pipeline (path+IK+grasp), and multi-biped coordination. Panels B, I, and J plot IK reach points and grasp contact points in yellow and blue dots, respectively. Our method generates feasible, collision-free, and task-consistent trajectories that satisfy geometric, kinematic, and interaction constraints across all settings.

Single Modality Motion Planning Tasks



Multi-Modality Motion Planning Tasks

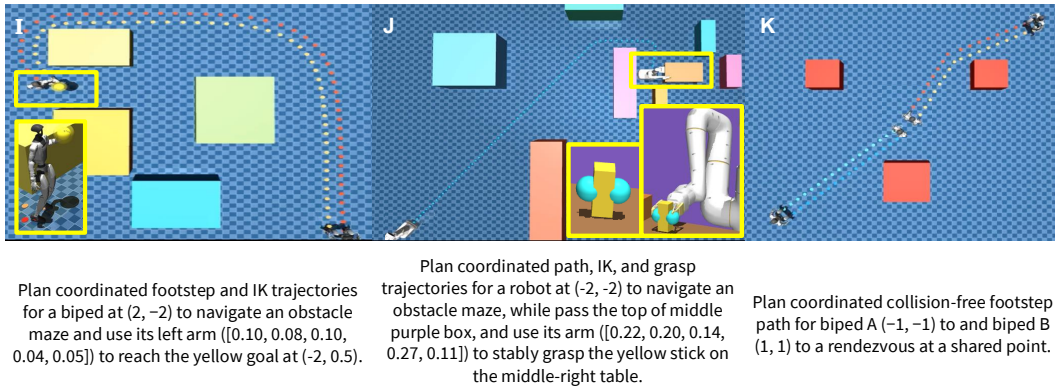
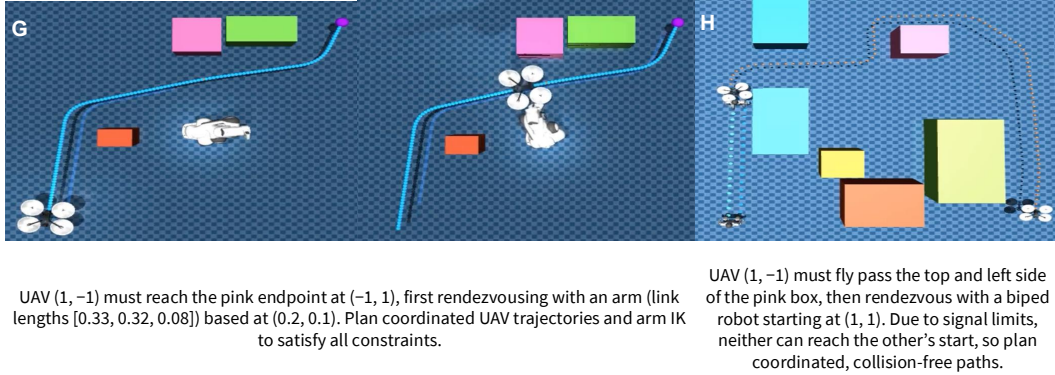
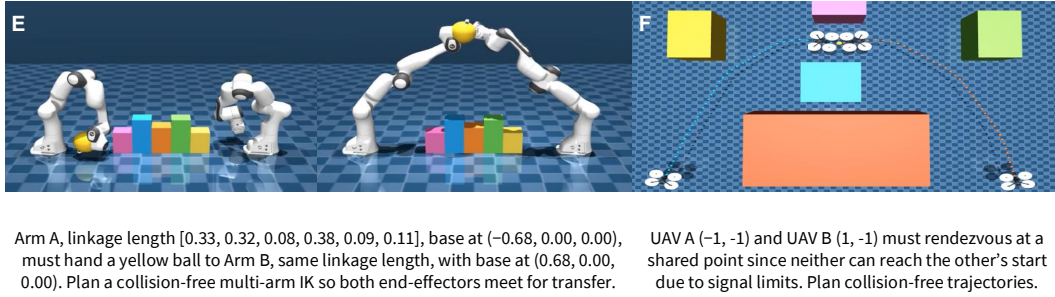
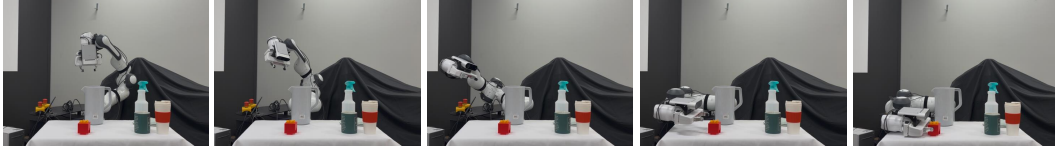


Figure 19: More qualitative results in MuJoCo simulation across single- and multi-modal M³P tasks. (A–D) Single-modal tasks: grasping, inverse kinematics (IK), UAV trajectory planning, and biped footstep planning. (E–K) Multi-modal tasks, including multi-arm coordination, multi-UAV rendezvous, UAV–arm cooperation, UAV–biped coordination, footstep+IK, full pipeline (path+IK+grasp), and multi-biped coordination. Our method generates feasible, collision-free, and task-consistent trajectories that satisfy geometric, kinematic, and interaction constraints across all settings.

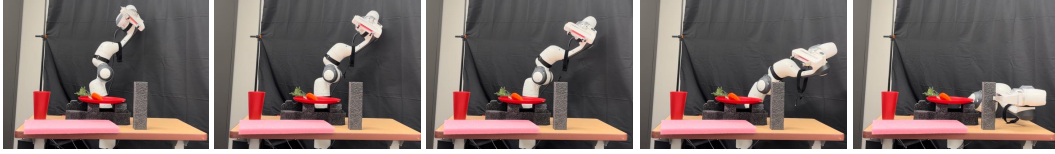


IK (with spatial): robot arm need to reach the red cubic from the left side of the gray pitcher



IK (without spatial): robot arm need to reach the red cubic

Figure 20: Real-world Franka Panda reaching task involving a red cube and a gray pitcher. Top: reaching the red cube with a spatial constraint relative to the gray pitcher. Bottom: reaching the red cube without the spatial constraint.

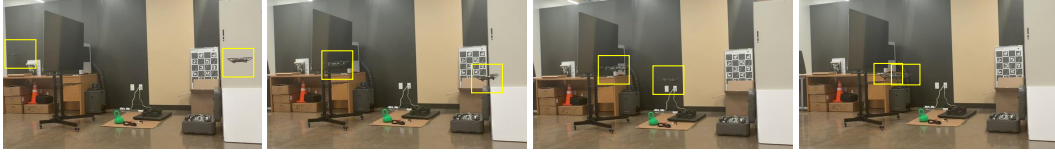


IK (with spatial): robot arm need to reach the gray rectangular beam from the right side of the red plate

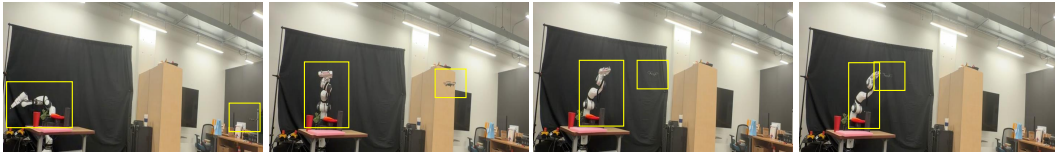


IK (without spatial): robot arm need to reach the gray rectangular beam

Figure 21: Real-world Franka Panda reaching task involving a gray rectangular beam and a red plate. Top: reaching the beam with an object-relative spatial constraint. Bottom: reaching the beam without the spatial constraint.



Multi-UAV: Left UAV must fly pass the left side of the television to meet right UAV at a shared point



UAV+IK: UAV cannot fly too far to the initial pose of robot arm, so plan the robot arm and the UAV to meet

Figure 22: Real-world composition results. Top: multi-UAV planning with an object-relative spatial constraint and a shared meeting point. Bottom: UAV+IK planning, where the UAV and robot arm are jointly planned to meet at a feasible shared point.

F LLM-Callable API

In this section, we list the set of LLM-callable APIs for discretizing the general non-convex constraints into disjointly convex sets.

F.1 Initialization APIs

Initialization APIs construct safe regions, optimization variables, and object representations. These functions define the problem structure before constraints are added.

```
generate_iris_regions(bounding_lb, bounding_ub, endpoints, num_regions,
obstacle_map)
```

Inputs. Workspace bounds, task endpoints (start/goal or base/target), number of regions, and obstacle map.

Description. Generates convex collision-free regions using IRIS. These regions serve as the feasible domain for all geometric constraints and enable efficient mixed-integer formulations.

```
augment_iris_regions(iris_regions, obstacle_map, styles, side_width)
```

Inputs. Base region set, obstacle metadata, spatial relations, and side width.

Description. Augments regions with auxiliary side boxes to encode spatial instructions such as directional passing or obstacle circling.

```
initialize_trajectory_problem(num_segments, num_regions)
```

Inputs. Number of trajectory segments and regions.

Description. Creates Bézier control points and binary region assignments, forming the core variables for trajectory optimization.

```
initialize_kinematic_problem(link_lengths, n_poly, num_intermediate,
base)
```

Inputs. Link lengths, rotation discretization level, collision sampling resolution, and base position.

Description. Initializes articulated chain variables, including pivots, rotations, and collision samples, for kinematic reasoning.

```
initialize_footstep_problem(num_regions, steps)
```

Inputs. Number of regions and planning horizon.

Description. Defines footstep positions, activation variables, and region assignments for biped locomotion.

```
initialize_finger_selection_problem(n_fingers, num_points)
```

Inputs. Number of fingers and candidate contact points.

Description. Initializes binary selection variables and robustness objective for contact selection without kinematics.

```
initialize_grasp_problem(link_lengths, n_poly, num_intermediate, base)
```

Inputs. Gripper link lengths, rotation discretization, collision resolution, and base.

Description. Initializes the kinematic chain that connects the robot base to grasp contact points.

```
build_finger_select_object(vertices_list)
```

Inputs. Object vertex list.

Description. Constructs geometric representation of the object used for contact sampling in finger selection.

```
sample_finger_contact_candidates(obj, delta, delta_n)
```

Inputs. Object geometry and spatial/normal sampling resolution.

Description. Samples candidate contact points and outward normals on the object surface.

```
build_grasp_object(vertices_list)
```

Inputs. Object vertices.

Description. Constructs object geometry for grasp planning, used for contact selection and collision reasoning.

F.2 Constraint APIs

Constraint APIs translate geometric, kinematic, and contact requirements into solver-verifiable constraints.

```
add_segment_assignment_constraints(H)
```

Inputs. Binary region assignment variables H .

Description. Ensures each segment selects exactly one region, enforcing discrete structure in trajectory planning.

```
add_region_membership_constraints(C, H, iris_regions, M)
```

Inputs. Points C , assignment variables H , regions, and Big- M (a large constant used for mixed-integer feasibility relaxation; see Appendix G).

Description. Constrains points to lie within assigned regions, linking discrete assignments to continuous geometry.

```
add_collision_free_constraints(points, iris_regions)
```

Inputs. Geometry samples and region set.

Description. Ensures all robot geometry remains in obstacle-free space through region assignment.

```
add_spatial_relation_constraints(H, styles)
```

Inputs. Assignment variables and spatial relations.

Description. Encodes high-level spatial instructions such as directional passing or obstacle circling.

```
add_trajectory_continuity_constraints(C)
```

Inputs. Control points C .

Description. Enforces smooth transitions between trajectory segments, ensuring physically realistic motion.

```
add_trajectory_endpoint_constraints(C, start, goal)
```

Inputs. Control points and endpoints.

Description. Anchors trajectory start and end positions to fixed or shared targets.

`add_reachability_constraints(positions)`

Inputs. Sequence of positions.

Description. Ensures transitions between consecutive states satisfy motion feasibility constraints.

`add_kinematic_constraints(pivots, rotations, link_lengths)`

Inputs. Joint pivots p , rotations R , and link lengths ℓ .

Description. Enforces forward kinematics, coupling joint configuration with spatial geometry.

`add_base_target_constraints(base_pivot, end_pivot, start, goal)`

Inputs. Base pivot, end-effector pivot, base position, and target position.

Description. Fixes the endpoints of a kinematic chain.

`add_contact_selection_constraints(selection)`

Inputs. Binary selection variables F .

Description. Selects valid and distinct contact points from candidates.

`add_end_effector_constraints(selection, points)`

Inputs. Selection variables and candidate contact points for grasp tasks.

Description. Maps selected contacts to end-effector positions.

`add_wrench_constraints(points, normals, friction_coef)`

Inputs. Contact points, normals, and friction coefficient μ .

Description. Enforces force-closure conditions in wrench space for stable grasping.

`add_objective(objective_terms)`

Inputs. Task-dependent objective terms.

Description. Defines the optimization objective, such as trajectory smoothness or grasp robustness.

G MIP Formulation

In this section, we provide the detailed formulation of each of our single-modal tasks.

Throughout the paper we use MIP as the umbrella term; the specific formulations below are mixed-integer quadratic programs (MIQPs) [18], and one variant is a mixed-integer semidefinite program (MISDP) [66].

Throughout this section, the modality subsections use *local notation*: symbols such as N , R , p_i , n_i , a_i , J , b , r , ρ , c_i , ℓ , σ_i , Λ , n (denoting reachability halfspace normals n_j and support-plane normals n_r), and the Cartesian coordinates y , z are locally redefined in each subsection (UAV, IK, Grasp, Finger Selection, Footstep), and across-modality reuse should be read as an abuse of notation with the meaning fixed by the subsection in which the symbol appears. By contrast, s and g always denote start and goal states inherited from the mission specification.

G.1 UAV

All symbols introduced below are scoped to this subsection unless stated otherwise. We formalize the UAV trajectory planning problem as an MIQP [18]. Let N denote the number of trajectory segments, R the number of IRIS regions, and $C \in \mathbb{R}^{4N \times 3}$ the cubic Bézier control points across all segments. We index $C_{j,k}$ for the k -th control point ($k = 0, 1, 2, 3$) of the j -th segment, and $C_{j,k,d}$ for

its d -th coordinate ($d \in \{x, y, z\}$). Binary variables $H_{r,j} \in \{0, 1\}$ assign segment j to region r . We aggregate the region-assignment, control-point feasibility (Big- M relaxed with region r described by $A_r x \leq b_r$ and $M > 0$ a sufficiently large constant), continuity, and start/goal constraints into a single labeled block:

$$\begin{aligned}
\sum_{r=1}^R H_{r,j} &= 1, & \forall j = 1, \dots, N, \\
A_r C_{j,k} &\leq b_r + M(1 - H_{r,j}), & \forall r, j, k, \\
C_{j,3} &= C_{j+1,0}, & \forall j = 1, \dots, N-1, \\
C_{j,3} - C_{j,2} &= C_{j+1,1} - C_{j+1,0}, & \forall j = 1, \dots, N-1, \\
C_{j,3} - 2C_{j,2} + C_{j,1} &= C_{j+1,0} - 2C_{j+1,1} + C_{j+1,2}, & \forall j = 1, \dots, N-1, \\
C_{0,0} &= s, \quad C_{N-1,3} = g.
\end{aligned} \tag{1}$$

The continuity rows are added by API call `add_continuity_constraints` and the start/goal row by `add_start_goal_constraints`.

Objective: We minimize the trajectory's integrated jerk, computed for each segment j and dimension $d \in \{x, y, z\}$ as:

$$J_{j,d} = 6(C_{j,3,d} - 3C_{j,2,d} + 3C_{j,1,d} - C_{j,0,d}).$$

The overall objective is then:

$$\min_{C,H} \sum_{j=1}^N \sum_{d \in \{x,y,z\}} J_{j,d}^2 \quad \text{s.t. Equation 1.} \tag{2}$$

The binary assignments H are integer feasibility variables and enter the program only through the region-assignment and Big- M rows of Equation 1; they do not appear in the objective body.

Equation 1 together with 2 define the UAV planning MIQP. This formulation tightly couples discrete region assignments with continuous Bézier control points, ensuring trajectories are collision-free, dynamically smooth, and solver-executable.

G.2 Extension to Our Novel Spatial Constraints

We propose a novel type of constraint to enforce additional constraints on UAV, enabling more expressive trajectory descriptions. Specifically, we allow UAV to fly from the “left/right/top/bottom/front/back” of an obstacle, for a given number of times. We further allow UAV to circle around the obstacle, also for a given number of times. These additional requirements can be achieved by so-called crossing constraints. Suppose there are two IRIS regions indexed by p and q , where each region is defined in $\mathbb{R}^3$ as $A_r x \leq b_r$, and we want to check whether UAV crosses the boundary of p and q at the a th trajectory segment. Such a check can be achieved by introducing the continuous indicator variable $X_{pq}^a \in [0, 1]$ and the following constraints:

$$\begin{aligned}
X_{pq}^a &\leq H_{p,a}, \\
X_{pq}^a &\leq H_{q,a+1}, \\
X_{pq}^a &\geq H_{p,a} + H_{q,a+1} - 1.
\end{aligned}$$

We can further ensure that only one crossing happens by the constraint $\sum_{a=1}^{N-1} (X_{pq}^a + X_{qp}^a) = 1$. Now, suppose we want the UAV to circle around an obstacle exactly once, then we can generate IRIS regions surrounding the obstacle, as indexed by $r_{\text{bot}}, r_{\text{rgt}}, r_{\text{top}}, r_{\text{lft}}$ (short for bottom, right, top, and left); we illustrate the construction in the horizontal plane, and the same idea generalizes to additional regions for fully 3D circling. We then enforce:

$$\sum_{a=1}^{N-1} X_{r_{\text{bot}} r_{\text{rgt}}}^a = \sum_{a=1}^{N-1} X_{r_{\text{rgt}} r_{\text{top}}}^a = \sum_{a=1}^{N-1} X_{r_{\text{top}} r_{\text{lft}}}^a = \sum_{a=1}^{N-1} X_{r_{\text{lft}} r_{\text{bot}}}^a = 1.$$

Further suppose we want the UAV to fly from the left of an obstacle. Then we can generate a rectangular IRIS region on the left, as indexed by r_{left} . We then create two other IRIS regions, as indexed $r_{\text{left,top}}$ and $r_{\text{left,bot}}$. $r_{\text{left,top}}$ intersects with r_{left} from the top and $r_{\text{left,bot}}$ intersects with r_{left} from the bottom. With these three IRIS regions, we only need to enforce that the UAV fly $r_{\text{left,top}}$ to r_{left} then to $r_{\text{left,bot}}$:

$$\sum_{a=1}^{N-1} X_{r_{\text{left,top}} r_{\text{left}}}^a = \sum_{a=1}^{N-1} X_{r_{\text{left}} r_{\text{left,bot}}}^a = 1.$$

G.3 IK

All symbols introduced below are scoped to this subsection unless stated otherwise. We present the simplified, 3D variant of Dai et al. [17], modeling a spatial manipulator with L links of fixed lengths $\{\ell_1, \dots, \ell_L\}$. Let $p_i \in \mathbb{R}^3$ denote the pivot of joint i , with base p_0 and end-effector p_L . Rotations are approximated via polyhedral discretization with N_{poly} facets. The MIP aggregates the following constraints. **Base anchoring** fixes the base at a given location s . **Rotation discretization**: each link's local-to-global transform uses a rotation matrix $\mathbf{R}_i \in \mathbb{R}^{3 \times 3}$ constrained to lie in a polyhedral approximation of $SO(3)$; each $\mathbf{R}_i$ is constructed via piecewise linear relaxation of $SO(3)$ by calling `add_chain_rotation_constraints`, a construction also used in footstep planning [21], yielding an outer approximation of $SO(3)$ with bounded approximation error that decreases with N_{poly} . The **end-effector constraint** forces the final position to coincide with the goal g . **Collision discretization** keeps each link collision-free (formulated similarly to the UAV case): each link $[p_{i-1}, p_i]$ is discretized into K intermediate points $q_{i,k}$, and all $\{p_i, q_{i,k}\}$ are concatenated into a collision-point matrix indexed by $j = 1, \dots, L(K+1) + 1$, with the j -th column denoted Q_j . We again construct N_R IRIS regions (in $\mathbb{R}^3$) with binary assignment variables $H_{r,j} \in \{0, 1\}$ and Big- M feasibility (safety distance d_{safe}), where each IRIS region r is described by $\{x \in \mathbb{R}^3 : A_r x \leq b_r\}$ (same convention as the UAV subsection) and $M > 0$ is a sufficiently large constant:

$$\begin{aligned} p_0 &= s, \\ p_i &= p_{i-1} + \mathbf{R}_i \begin{bmatrix} \ell_i \\ 0 \\ 0 \end{bmatrix}, & i = 1, \dots, L, \\ p_L &= g, \\ q_{i,k} &= \left(1 - \frac{k}{K+1}\right)p_{i-1} + \frac{k}{K+1}p_i, & k = 1, \dots, K, \\ \sum_{r=1}^{N_R} H_{r,j} &= 1, & \forall j = 1, \dots, L(K+1) + 1, \\ A_r Q_j &\leq (b_r - d_{\text{safe}}) + M(1 - H_{r,j}), & \forall r, j = 1, \dots, L(K+1) + 1. \end{aligned} \tag{3}$$

The constraints in Equation 3 define our MIQP. The formulation couples discrete rotation approximations, continuous kinematics, and region-based collision avoidance.

G.4 Grasp

All symbols introduced below are scoped to this subsection unless stated otherwise. We present a simplified, 3D variant of [15] for grasp planning. The grasp formulation extends the IK problem introduced in the previous section. All IK constraints (chain kinematics, collision discretization, IRIS region assignment, and optional spatial relation constraints) are inherited directly. We now introduce additional variables and constraints that model grasp contact selection and force-closure feasibility.

Grasp selection (two fingers): We sample N equidistant points on the surface of objects as candidate grasp points $\{c_i\}$, which is achieved by calling `get_object_surface_samples`. We further introduce binary matrices $F \in \{0, 1\}^{2 \times N}$ and binary selection flags $\chi \in \{0, 1\}^N$, where $\chi_i = 1$ iff candidate i is selected, that select two distinct contact points from the N surface samples, via the

following constraints:

$$\begin{aligned} \sum_{i=1}^N F_{1i} &= 1, & \sum_{i=1}^N F_{2i} &= 1, \\ F_{1i} + F_{2i} &\leq 1, & \chi_i &= F_{1i} + F_{2i}, \quad i = 1, \dots, N. \end{aligned}$$

The selected end-effector (contact) positions are:

$$\mathbf{ee}_1 = \sum_{i=1}^N F_{1i} c_i, \quad \mathbf{ee}_2 = \sum_{i=1}^N F_{2i} c_i.$$

The above two variables replace the fixed start and goal constraints from the IK problem by anchoring the kinematic chain to two contact points on the object.

Force-closure (wrench) constraints: For each candidate contact $c_i \in \mathbb{R}^3$ with outward unit normal n_i and two orthonormal tangent directions $\mathbf{t}_i^{(1)}, \mathbf{t}_i^{(2)}$, we define a polyhedral approximation of the friction cone with friction coefficient μ using four generators:

$$\begin{aligned} f_i^{(1)} &= n_i + \mu \mathbf{t}_i^{(1)}, \\ f_i^{(2)} &= n_i - \mu \mathbf{t}_i^{(1)}, \\ f_i^{(3)} &= n_i + \mu \mathbf{t}_i^{(2)}, \\ f_i^{(4)} &= n_i - \mu \mathbf{t}_i^{(2)}. \end{aligned}$$

with corresponding spatial wrenches:

$$w_i^{(k)} = \begin{bmatrix} f_i^{(k)} \\ c_i \times f_i^{(k)} \end{bmatrix} \in \mathbb{R}^6, \quad k = 1, \dots, 4.$$

Let $\{\hat{w}_d\}_{d=1}^D \subset \mathbb{R}^6$ be a fixed set of D test wrench directions, and let $\rho \geq 0$ denote the common inscribed sphere radius in wrench space (the grasp robustness margin to be maximized below). Introducing nonnegatives $\alpha_{i,k,d}$ and slacks γ_d for each test wrench direction, we impose:

$$\begin{aligned} \sum_{i=1}^N \sum_{k=1}^4 w_i^{(k)} \alpha_{i,k,d} &= \gamma_d \hat{w}_d, \quad \forall d = 1, \dots, D. \\ \sum_{k=1}^4 \alpha_{i,k,d} &\leq 1, \quad \forall i, d, \\ \sum_{k=1}^4 \alpha_{i,k,d} &\leq \chi_i, \quad \forall i, d, \\ \alpha_{i,k,d} &\geq 0, \quad \forall i, k, d, \\ \gamma_d &\geq \rho, \quad \forall d = 1, \dots, D. \end{aligned}$$

A 4-generator pyramid yields a coarse but sufficient polyhedral approximation of the friction cone, capturing its convex geometry while maintaining a minimal number of linear constraints for efficient MIQP optimization.

Objective: Our objective is to maximize the inscribed sphere radius:

$$\max_{\rho, F, \chi, \alpha, \gamma} \rho \quad \text{s.t. (force-closure and selection constraints above).}$$

Here F, χ, α, γ are auxiliary variables constrained by the equations above. Note that this inscribed sphere radius can be approximated using a discretization technique [15], leading to an MIQP, or the lower-bound relaxation [66], leading to an MISDP. We use the MIQP formulation since Gurobi does not solve MISDP. Put together, the grasp MIP augments the IK formulation with the above constraints, which jointly enforce valid finger selection and force-closure robustness.

G.5 Finger Selection

All symbols introduced below are scoped to this subsection unless stated otherwise. Finger selection can be viewed as a simplified version of the grasp problem (Appendix G.4), where we do not consider the reachability of fingers, and directly select k contact points from N sampled candidate grasp points to maximize the inscribed sphere radius, where $k \leq N$ is a user-specified contact count.

G.6 Footstep

All symbols introduced below are scoped to this subsection unless stated otherwise. Footstep planning [21] builds upon the general spatial and region constraints introduced above, but specializes them for biped locomotion.

Let N denote the maximum number of footsteps. Each step $i = 0, \dots, N-1$ has a contact position $p_i = (x_i, y_i, z_i) \in \mathbb{R}^3$. Although the decision variable is written in $\mathbb{R}^3$, feasible contacts lie on locally planar support surfaces, yielding a 2.5D terrain representation while retaining height variation. Let R denote the number of convex safe regions generated by IRIS for the footstep problem. Let $a_i \in \{0, 1\}$ indicate whether step i is active, $H_{r,i} \in \{0, 1\}$ indicate whether step i is assigned to convex safe region $r = 1, \dots, R$, and $\delta_i^{\text{term}} \in \{0, 1\}$ indicate whether step i is the terminal footstep. We use $M > 0$ as a sufficiently large big- M constant.

Region membership. Let $\{p \in \mathbb{R}^3 : A_r p \leq b_r\}$ denote the collision-free convex region generated by IRIS. Each active footstep must lie in exactly one convex safe region:

$$\begin{aligned} \sum_{r=1}^R H_{r,i} &= a_i, & \forall i, \\ A_r p_i &\leq b_r + M(1 - H_{r,i})\mathbf{1}, & \forall r, i. \end{aligned}$$

The trajectory between consecutive footsteps is not constrained to remain inside these regions, because the robot can lift its swing foot over obstacles.

Support-surface constraints. For 3D or 2.5D terrain, region membership alone does not guarantee that a footstep lies on a valid contact surface. For each support region r , let $\Pi_r = \{p \in \mathbb{R}^3 : n_r^\top p = h_r\}$ denote the local support plane, where $n_r \in \mathbb{R}^3$ is the plane normal and $h_r \in \mathbb{R}$ is the plane offset. We activate the support-plane constraint using the same region-assignment variable:

$$-\epsilon_z - M(1 - H_{r,i}) \leq n_r^\top p_i - h_r \leq \epsilon_z + M(1 - H_{r,i}), \quad \forall r, i,$$

where $\epsilon_z \geq 0$ is a small contact-height tolerance. For flat-ground experiments, this reduces to $z_i = 0$ for all active footsteps.

Start and terminal constraints. The first two contacts are fixed to the initial left and right foot positions, $p_0 = p_L^{\text{start}}$ and $p_1 = p_R^{\text{start}}$, and are always active, $a_0 = a_1 = 1$. Exactly one active step is selected as the terminal footstep:

$$\sum_{i=0}^{N-1} \delta_i^{\text{term}} = 1, \quad \delta_i^{\text{term}} \leq a_i, \quad \forall i.$$

We introduce an auxiliary terminal position $p^{\text{term}} \in \mathbb{R}^3$ and enforce:

$$-M(1 - \delta_i^{\text{term}})\mathbf{1} \leq p^{\text{term}} - p_i \leq M(1 - \delta_i^{\text{term}})\mathbf{1}, \quad \forall i.$$

Thus, if $\delta_i^{\text{term}} = 1$, then $p^{\text{term}} = p_i$.

Step activation monotonicity. Once a step is inactive, no later step can be active: $a_i \geq a_{i+1}$ for all $i = 0, \dots, N-2$. This constraint can be added by calling `add_monotonicity_constraints`.

Left-right ordering and lateral separation. Footsteps alternate between left and right feet. Let $\sigma_i \in \{-1, +1\}$ denote the side of step i , where $\sigma_i = +1$ represents the left foot and $\sigma_i = -1$ represents the right foot. Let $e_{\text{lat}} \in \mathbb{R}^3$ be the lateral unit direction of the walking frame, pointing from the right side to the left side, and let $\Delta_{\min} > 0$ be the minimum lateral separation between consecutive feet. For each active transition, we impose:

$$\sigma_{i+1} e_{\text{lat}}^\top (p_{i+1} - p_i) \geq \Delta_{\min} - M(1 - a_{i+1}), \quad \forall i = 0, \dots, N-2.$$

This enforces each active footstep to lie on the correct lateral side of the previous stance foot. In a planar setting where x is the lateral coordinate, this reduces to the corresponding signed constraint on $x_{i+1} - x_i$.

Reachability constraints. From stance foot p_i , the next contact p_{i+1} must lie in a shifted reachable polytope. Let the local reachable set be approximated by N_J halfspaces with normals $n_j \in \mathbb{R}^3$ and offsets $b_j^{\text{reach}} \in \mathbb{R}$:

$$\mathcal{R}_{\text{reach}} = \{q \in \mathbb{R}^3 : n_j^\top q \leq b_j^{\text{reach}}, j = 1, \dots, N_J\}.$$

Let $\Delta_i^{\text{nom}} \in \mathbb{R}^3$ denote the nominal offset from stance foot i to the next footstep, which may depend on the left/right side. The reachability constraint is:

$$n_j^\top (p_{i+1} - p_i - \Delta_i^{\text{nom}}) \leq b_j^{\text{reach}} + M(1 - a_{i+1}), \quad \forall i = 0, \dots, N-2, j = 1, \dots, N_J.$$

Goal-error constraints. Let $p_g = (x_g, y_g, z_g) \in \mathbb{R}^3$ denote the desired terminal footstep location. We introduce nonnegative slack variables $\varepsilon_k \geq 0$ for $k \in \{x, y, z\}$ to measure terminal goal error:

$$\varepsilon_k \geq p_k^{\text{term}} - p_{g,k}, \quad \varepsilon_k \geq -(p_k^{\text{term}} - p_{g,k}), \quad k \in \{x, y, z\}.$$

At optimum, $\sum_{k \in \{x, y, z\}} \varepsilon_k = \|p^{\text{term}} - p_g\|_1$.

Nominal-lane deviation constraints. We define a nominal walking lane to regularize footstep placement. Let $\bar{p}_i \in \mathbb{R}^3$ be the nominal centerline point for step i , $\hat{n}_i^{\text{lat}} \in \mathbb{R}^3$ be the local lateral unit direction, and $w_s > 0$ be the nominal half-stance width. The nominal footstep location is $p_i^{\text{nom}} = \bar{p}_i + \sigma_i w_s \hat{n}_i^{\text{lat}}$. We define d_i as the activated ℓ_1 deviation of step i from this nominal location. For each coordinate $k \in \{x, y, z\}$, introduce a nonnegative slack variable $d_{i,k} \geq 0$ and impose:

$$\begin{aligned} d_{i,k} &\geq p_{i,k} - p_{i,k}^{\text{nom}} - M(1 - a_i), & \forall i, k, \\ d_{i,k} &\geq -(p_{i,k} - p_{i,k}^{\text{nom}}) - M(1 - a_i), & \forall i, k, \\ 0 &\leq d_{i,k} \leq M a_i, & \forall i, k. \end{aligned}$$

Then $d_i = \sum_{k \in \{x, y, z\}} d_{i,k}$. Thus, $d_i = 0$ for inactive footsteps, while for active footsteps it equals the ℓ_1 deviation from the preferred left/right nominal lane location at optimum.

Step-length constraints. To linearize the ℓ_1 distance between consecutive active footsteps, introduce nonnegative slack variables $\Lambda_{i,k} \geq 0$ for each transition $i = 0, \dots, N-2$ and coordinate $k \in \{x, y, z\}$:

$$\begin{aligned} \Lambda_{i,k} &\geq p_{i+1,k} - p_{i,k} - M(1 - a_{i+1}), & \forall i, k, \\ \Lambda_{i,k} &\geq -(p_{i+1,k} - p_{i,k}) - M(1 - a_{i+1}), & \forall i, k, \\ 0 &\leq \Lambda_{i,k} \leq M a_{i+1}, & \forall i, k. \end{aligned}$$

We define $\Lambda_i = \sum_{k \in \{x, y, z\}} \Lambda_{i,k}$. Thus, $\Lambda_i = 0$ when the transition to step $i+1$ is inactive, and $\Lambda_i = \|p_{i+1} - p_i\|_1$ at optimum when the transition is active.

Objective. The footstep planner minimizes a weighted sum of step length, number of active footsteps, terminal goal error, and nominal-lane deviation:

$$\begin{aligned} \min_{\{p_i, a_i, H_{r,i}, \delta_i^{\text{term}}, p^{\text{term}}, \varepsilon_k, d_{i,k}, \Lambda_{i,k}\}} \quad & w_1 \sum_{i=0}^{N-2} \Lambda_i + w_2 \sum_{i=0}^{N-1} a_i \\ & + w_3 \sum_{k \in \{x,y,z\}} \varepsilon_k + w_4 \sum_{i=0}^{N-1} d_i \\ \text{s.t.} \quad & \text{above constraints,} \end{aligned}$$

where $w_1, w_2, w_3, w_4 \geq 0$ are user-defined weights. The first term encourages short steps, the second encourages using fewer footsteps, the third penalizes terminal goal error, and the fourth keeps active footsteps close to their preferred left/right nominal lane locations. The binary variables a_i , $H_{r,i}$, and δ_i^{term} enter through the feasibility constraints, while the continuous variables p_i and p^{term} determine the step locations and terminal contact.

G.7 Multi-modal Tasks

All symbols introduced below are scoped to this subsection unless stated otherwise. In multi-modal tasks, subsystems such as UAV, IK, and Finger Selection must coordinate at specific interface points—for example, the UAV endpoint aligning with the IK base, or the IK end-effector aligning with the Grasp target. These *shared meeting points* enforce cross-domain consistency while preserving feasibility within each individual modality.

Rather than relying on predefined coupling mechanisms, we train the LLM via GRPO-based reinforcement learning to autonomously discover and construct such coordination strategies. In particular, the model learns to introduce shared variables and synthesize cross-domain constraints that couple multiple modalities within a unified optimization program, yielding jointly consistent and solver-feasible multi-modal plans.

H Method Implementation Detail

Here we describe the fine-tuning details of our methods and implementation of baselines.

H.1 Fine-tuning Detail

We fine-tune the model in two stages. Stage 1 uses SFT to teach the model the required output format, discretization API usage, and executable MIP-program structure. We use Low-Rank Adaptation (LoRA) [67], which freezes the base model weights and updates only the adapter parameters. Stage 2 further optimizes the SFT model with GRPO using solver-grounded verifiable rewards. During both stages, the base model weights are frozen and only the LoRA adapter parameters are updated, which makes the training more memory- and compute-efficient. The LoRA adapter uses rank $r = 16$, `lora.alpha` = 32, and is applied to `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj` modules.

Because our rewards are solver-grounded and directly verifiable, policy updates are guided by deterministic execution and feasibility signals rather than noisy human-preference feedback. We therefore follow recent RLVR practice [68, 69] and disable KL regularization by setting the KL coefficient $\beta = 0.0$. This design avoids over-constraining the policy toward the SFT initialization and allows broader exploration during solver-grounded optimization. Similar choices have been adopted in recent reasoning-oriented RL systems, such as Decoupled Clip and Dynamic sAmpling Policy Optimization (DAPO) [68] and Understanding R1-Zero-Like Training [69], which report a zero KL coefficient in their training configuration.

The complete two-stage fine-tuning process was conducted on one NVIDIA H100 80GB GPU for approximately 30 hours. At inference time, the M³P-R1 policy is served on a single NVIDIA L4

Table 13: Stage 1 supervised fine-tuning details.

Parameter	Value
Maximum training epoch	1
Learning rate	2×10^{-4}
Per-device batch size	2
Gradient accumulation steps	4
Effective batch size	8 per GPU process
Maximum sequence length	8192
Optimizer	adamw
Learning-rate scheduler	Linear
Warmup ratio	0.1
Weight decay	0.01

Table 14: Stage 2 GRPO fine-tuning details.

Parameter	Value
Maximum training steps	800
Per-device batch size	4
Number of generations per prompt	4
Maximum sequence length	8192
Learning rate	2×10^{-6}
KL coefficient	$\beta = 0.0$
Gradient accumulation steps	1
Optimizer	adamw
Learning-rate scheduler	Linear
Warmup ratio	0.1
Weight decay	0.001
Sampling temperature	0.6
Sampling min_p	0.1
Sampling top_p	0.95
Sampling top_k	50
Precision	bf16
Stop strings	</call> and end-of-sequence (EOS) token
Reward weights	$\lambda_1 = 1, \lambda_2 = 2, \lambda_3 = 1, \lambda_4 = 1$

GPU with vLLM [70]; the average end-to-end LLM call latency is approximately 4.5 s per task at $T_{\text{samp}} = 0.6$ sampling (range: 2.1–11.4 s depending on output length). The detailed SFT and GRPO configurations are summarized in Table 13 and 14.

H.2 Baselines Setup Detail

LLM-baseline evaluation protocol. For each prompt-only LLM baseline reported in Table 1 and 2, as well as for the LLM component of LLM+P, LLM+RRT*, LLM+BIT*, and LLM+TrajOpt, we use **GPT-5.2** as the underlying LLM. Sampling uses temperature $T_{\text{samp}} = 0.6$, a maximum output length of 8192 tokens (matched to the training context window in Table 14), and one in-context example for format alignment. On parse failure (e.g., missing or malformed <think>/<code> block), we retry the call once before recording the trial as a failure. The GPT-5.2 API endpoint version is pinned to the evaluation date so that all runs in this paper use the same underlying model snapshot. ReAct, in contrast, uses Qwen3-8B as the backbone with the same callable API handbook provided in context (see the ReAct paragraph below). For all settings, the same API handbook, mission instruction, environment JSON, and obstacle map are supplied in the prompt; the only quantity that varies across runs of the same baseline is the sampling seed.

PDDLStream. We adopt PDDLSTREAM as a symbolic–geometric planning baseline for M³P. This baseline assumes a fully grounded symbolic specification and does not perform language understanding. In our setup, the environment is parsed from the structured JSON block embedded in `mission_instruction.txt`, where task semantics (e.g., obstacle-side constraints) are already encoded via `obstacle_map[...]["style"]`. These styles are deterministically compiled into discrete tasks and geometric variants, while continuous feasibility (collision checking and distance evaluation) is handled via external streams over a visibility roadmap. Planning is performed using an incremental solver that jointly selects task variants and motion transitions under a distance-based cost objective. Key parameters include roadmap collision padding (10^{-7}), node margin (10^{-3}), fixed style-region width (e.g., 0.2), deterministic milestone sampling, and a planning timeout of 60s.

LLM+P. To enable language grounding, we introduce an LLM-based preprocessing layer that translates free-form mission instructions into a structured symbolic representation. The model takes as input the natural-language instruction and environment JSON, and outputs a constrained JSON intermediate representation that specifies task-relevant obstacles, normalized styles, ordering, and (for multi-agent tasks) coordination variables such as shared meeting points. This IR is strictly validated and deterministically compiled into a PDDLStream-ready problem by assigning styles to `obstacle_map`, enforcing task ordering, and filtering geometric variants. Planning is then performed using the same PDDLStream backend with an augmented domain that enforces the LLM-specified structure. We use deterministic decoding (temperature 0.1) and enforce a fixed schema with restricted style and variant vocabularies. While this baseline enables language-to-symbolic grounding, intermediate variables are generated heuristically and are not jointly optimized with geometric constraints, which can lead to infeasible or suboptimal solutions.

Expert-MIP. To determine whether M³P-R1’s performance can be explained solely by access to high-quality expert optimization modules, we construct an Expert-MIP baseline. Given the natural-language task and environment description, an LLM first identifies the required planning modalities, selects the corresponding fixed expert MIP modules, and predicts the interface variables needed to connect them. These interfaces include quantities such as shared meeting points, contact locations, target poses, and intermediate states. Each expert module then solves its corresponding subproblem using a hand-designed formulation; the LLM does not generate or modify the internal optimization constraints. We evaluate this baseline using both Qwen3-8B and GPT-5.2. The expert modules achieve 100.0% success on single-modal tasks, but multi-modal success decreases to 55.6% and 63.2%, respectively, because an interface predicted independently of downstream geometric feasibility can render subsequent IK, grasp, or motion-planning stages infeasible. This baseline therefore isolates the benefit of learning coupled optimization formulations beyond selecting and composing fixed expert modules.

LLM+RRT* / LLM+BIT*. We implement sampling-based motion planning baselines by combining LLM-predicted waypoint decomposition with classical planners in $\mathbb{R}^3$. The LLM takes as input the mission instruction and environment description, and predicts a sequence of key waypoints and their order to satisfy style constraints (e.g., passing above/below obstacles) and coordination requirements (e.g., meeting points). Each consecutive waypoint pair defines a planning segment, for which a collision-free path is computed using OMPL planners [65], specifically RRT* or BIT*. The state space is a bounded 3D Euclidean space, and collision checking rejects states inside axis-aligned obstacles inflated by a fixed clearance margin. Planning is performed independently per segment, and resulting paths are concatenated to form the final trajectory. Planner-specific hyperparameters follow OMPL defaults. This baseline evaluates LLM-based task decomposition with sampling-based feasibility, but the decoupled waypoint generation and segment-wise planning can lead to suboptimal or infeasible trajectories under complex multi-modal constraints.

LLM+TrajOpt. We include a trajectory optimization baseline using TRAJOPT [41]. The LLM first predicts waypoint sequences from the instruction and environment. Initial trajectories are generated by connecting waypoints with BIT*, and then refined using TrajOpt, which performs sequen-

Table 15: Capability comparison. Lang.: language grounding; Feas.: continuous feasibility; Joint: joint discrete-continuous optimization; Multi: multi-modal coordination; Opt.: solution optimality.

Method	Lang.	Feas.	Joint	Multi	Opt.
PDDLStream	No	Yes	Partial	Limited	Approx
LLM+P	Yes	Yes	No	Weak	Subopt
Ours (MIP)	Yes	Yes	Yes	Yes	Near-opt

tial convex optimization over a discretized trajectory to minimize smoothness and collision costs. Obstacles are modeled as inflated axis-aligned boxes (clearance ≈ 0.2), and optimized segments are concatenated and densified (e.g., $\Delta s = 0.03$). This improves local smoothness but remains dependent on LLM-generated waypoints and does not achieve global joint optimization.

ReAct. We choose Qwen3-8B as the backbone model and also provide the same callable API handbook we used for RFT to Qwen3-8B. ReAct is an agentic workflow that iteratively performs reasoning and tool invocation to solve the task. At each step, the model generates an action (e.g., calling planning or geometry APIs) based on intermediate observations, forming a reasoning-acting loop until a solution is produced or a fixed time budget is reached (180s). However, without fine-tuning, LLMs lack the skills for MIP discretizations and long-horizon reasoning, which often leads to failure under complex multi-modal constraints.

Table 15 summarizes the capability comparison.

Scope. The sampling-based baselines (LLM+RRT*, LLM+BIT*, and LLM+TrajOpt) are designed for geometric path planning and are not directly applicable to tasks involving discrete contact sequences or kinematic feasibility (e.g., footstep planning, grasp synthesis, or inverse kinematics). Therefore, we evaluate these baselines only on single-agent and multi-agent UAV navigation tasks.

Comparison to ours. In contrast, our method directly generates solver-grounded MIP programs and jointly optimizes continuous trajectories and discrete mode transitions within a unified optimization framework, ensuring global feasibility and consistency.

I Additional Related Work

Data-driven methods for MIP. Beyond hand-designed MIP formulations, recent work has explored learning-based techniques to accelerate mixed-integer optimization, including learned branching heuristics [71], node selection [72], and cutting-plane selection [73]. These methods improve solver efficiency but typically assume that the MIP instance is already specified. Several datasets and environments, such as Ecole [74], support benchmarking machine-learning methods for combinatorial optimization. However, these benchmarks primarily focus on exact MIP/MILP instances rather than approximate, geometry-specific formulations used in robot motion planning. In contrast, M³P-R1 addresses the problem of generating executable approximate MIP formulations from natural-language task descriptions, where the model must choose and compose discretization primitives rather than merely accelerate an existing solver.

LLM-based planning variants. LLMs have been used for planning in several forms. Direct prompting methods generate high-level action sequences from natural-language goals [22, 23, 24, 25], but their outputs are often difficult to verify. ReAct-style methods [43] interleave reasoning and acting, improving interactive problem solving but still lacking formal guarantees on feasibility or constraint satisfaction. LLM+P frameworks [26, 27, 28, 44] translate language into symbolic planning representations such as PDDL, which are then solved by classical planners. These methods benefit from symbolic structure but are limited by predefined abstractions and require additional mechanisms for continuous feasibility. Code-generation approaches such as Code as Policies [45]

and ProgPrompt [46] produce executable robot programs, but generally output open-loop code rather than solver-verified optimization formulations.

LLM-guided continuous planning. A related class of methods uses LLMs to propose waypoints, subgoals, or intermediate states, followed by classical continuous planners such as RRT* [38], BIT* [39], or trajectory optimizers such as TrajOpt [41]. These approaches are useful when a high-level language instruction can be decomposed into waypoint targets, but the symbolic decomposition and continuous optimization are still decoupled. As a result, they can struggle when task success depends on jointly optimizing shared meeting points, contact interfaces, reachability constraints, or synchronized multi-agent states. Our LLM+MIP formulation instead generates a single solver-executable program in which discrete mode choices and continuous trajectories are optimized together.

LLMs for optimization modeling. Recent work has also explored using LLMs for optimization tasks, including diagnosing infeasible optimization models [52], generating supply-chain or operations-research formulations [51], evolving heuristics [53], translating natural language into MIP scripts [35, 36], and building foundation models for MILP [37]. These works primarily target exact MIP/MILP problems, where the formulation structure is often algebraic and domain-independent. M³P-R1 differs by targeting approximate MIP for robotics, where valid formulations require geometry-specific discretization, collision constraints, reachability approximations, and cross-modal coupling constraints.

Solver-grounded reinforcement fine-tuning. Reinforcement fine-tuning has become a common strategy for improving structured reasoning in LLMs. RLHF [54], PPO [55], DPO [56], and GRPO [57] optimize models using outcome- or preference-based rewards. In verifiable domains, external tools can provide stronger supervision: proof assistants such as Lean validate mathematical reasoning [75, 76], symbolic solvers support geometry reasoning [77, 78], and interpreters or compilers verify generated programs [59, 60]. Solver-Informed RL [58] further demonstrates the value of solver feedback for optimization modeling. M³P-R1 follows this solver-grounded paradigm, but applies it to language-conditioned robotic M³P, where rewards evaluate not only code execution but also solver feasibility, task satisfaction, and multi-modal trajectory correctness.

J LLMM³P Bench Data Example

Here we showcase both single- and multi-modal motion planning data examples generated for our dataset.

UAV Trajectory Planning

Input instruction.

Begin the flight mission at the start position (1.8, 0.3). First, navigate to the left side of the bottom obstacle located between (0.0, −1.5) and (0.9, −0.7). Next, approach the top side of the upper obstacle spanning from (−1.3, 0.0) to (−0.3, 0.7). Finally, smoothly proceed to the goal position (−0.4, 2.0) within the bounded workspace $[-3.0, 3.0] \times [-3.0, 3.0]$.

Generated executable program.

```
1 # Environment and task setup
2 bounding_lb = [-3.0, -3.0]
3 bounding_ub = [3.0, 3.0]
4 start_pos = [1.8, 0.3]
5 goal_pos = [-0.4, 2.0]
6
7 num_segments = 12
8 num_regions = 32
9 big_M = 10.0
10 side_width = 0.2
11
```

```

12 obstacle_map = {
13     "obst1": {
14         "lb": [[0.0], [-1.5]],
15         "ub": [[0.9], [-0.7]],
16         "style": "bottom",
17     },
18     "obst2": {
19         "lb": [[-1.3], [0.0]],
20         "ub": [[-0.3], [0.7]],
21         "style": "top",
22     },
23 }
24
25 # Region generation
26 endpoints = [start_pos, goal_pos]
27 iris_regions, obstacles_list_plt, styles, obstacle_keys = generate_iris_regions(
28     bounding_lb,
29     bounding_ub,
30     endpoints,
31     num_regions,
32     obstacle_map,
33 )
34
35 iris_regions, styles = augment_iris_regions(
36     iris_regions,
37     obstacle_map,
38     styles,
39     side_width,
40 )
41
42 # Problem initialization
43 C, H, var_dict = initialize_trajectory_problem(
44     num_segments,
45     len(iris_regions),
46 )
47
48 # Constraints
49 constraints = []
50 constraints += add_segment_assignment_constraints(H)
51 constraints += add_region_membership_constraints(C, H, iris_regions, big_M)
52 constraints += add_spatial_relation_constraints(H, styles)
53 constraints += add_trajectory_continuity_constraints(C)
54 constraints += add_trajectory_endpoint_constraints(C, start_pos, goal_pos)
55
56 # Objective and solve
57 objective = add_objective(["trajectory_smoothness"])
58 status, cost, solution = solve_problem(
59     constraints,
60     objective,
61     var_dict,
62 )

```

Inverse Kinematics Planning

Input instruction.

The mission is for a robotic arm with five links of lengths 0.9, 1.0, 0.4, 0.5, and 0.6 m to move from the base position $(-1.6, -1.4)$ to the end-effector target $(0.6, 0.7)$. While navigating, avoid the top obstacle extending from $(-0.1, -0.2)$ to $(0.7, 0.7)$ and the secondary obstacle from $(0.2, -1.3)$ to $(1.2, -0.6)$.

Generated executable program.

```

1 # Environment and task setup
2 bounding_lb = [-2.0, -2.0]
3 bounding_ub = [2.0, 2.0]
4 base_location = [-1.6, -1.4]
5 target_location = [0.6, 0.7]
6
7 link_lengths = [0.9, 1.0, 0.4, 0.5, 0.6]
8 n_poly = 8
9 num_intermediate = 10
10 num_regions = 20
11 big_M = 10.0

```

```

12 side_width = 0.2
13
14 obstacle_map = {
15     "obst1": {
16         "lb": [[-0.1], [-0.2]],
17         "ub": [[0.7], [0.7]],
18         "style": "top",
19     },
20     "obst2": {
21         "lb": [[0.2], [-1.3]],
22         "ub": [[1.2], [-0.6]],
23         "style": "none",
24     },
25 }
26
27 # Region generation
28 endpoints = [base_location, target_location]
29 iris_regions, obstacles_list_plt, styles, obstacle_keys = generate_iris_regions(
30     bounding_lb,
31     bounding_ub,
32     endpoints,
33     num_regions,
34     obstacle_map,
35 )
36
37 iris_regions, styles = augment_iris_regions(
38     iris_regions,
39     obstacle_map,
40     styles,
41     side_width,
42 )
43
44 # Problem initialization
45 pivots, rotations, collision_points, var_dict = initialize_kinematic_problem(
46     link_lengths,
47     n_poly,
48     num_intermediate,
49     base_location,
50 )
51
52 # Constraints
53 constraints = []
54 constraints += add_base_target_constraints(
55     pivots[0],
56     pivots[-1],
57     base_location,
58     target_location,
59 )
60 constraints += add_kinematic_constraints(
61     pivots,
62     rotations,
63     link_lengths,
64 )
65 constraints += add_collision_free_constraints(
66     collision_points,
67     iris_regions,
68 )
69 constraints += add_spatial_relation_constraints(
70     var_dict["H"],
71     styles,
72 )
73
74 # Objective and solve
75 objective = add_objective(["feasibility"])
76 status, cost, solution = solve_problem(
77     constraints,
78     objective,
79     var_dict,
80 )

```

Footstep Planning

Input instruction.

Plan a biped locomotion path from the initial left foot position (0.0,0.0) and right foot position (0.5,0.0) to the goal position (14.0,14.0) while avoiding obstacles in the workspace $[-5.0, 15.0] \times [-5.0, 15.0]$.

Generated executable program.

```

1  # Environment and task setup
2  bounding_lb = [-5.0, -5.0]
3  bounding_ub = [15.0, 15.0]
4
5  start_left = [0.0, 0.0]
6  start_right = [0.5, 0.0]
7  goal_pos = [14.0, 14.0]
8
9  steps = 15
10 step_dist = 1.5
11 num_regions = 12
12 big_M = 10.0
13
14 obstacle_map = {
15     "obst1": {
16         "lb": [-0.2, 11.3],
17         "ub": [2.1, 13.1],
18         "style": "none",
19     },
20     "obst2": {
21         "lb": [2.9, 1.3],
22         "ub": [5.4, 3.1],
23         "style": "none",
24     },
25     "obst3": {
26         "lb": [11.0, -1.9],
27         "ub": [12.7, -0.4],
28         "style": "none",
29     },
30 }
31
32 # Region generation
33 endpoints = [start_left, start_right, goal_pos]
34 iris_regions, obstacles_list_plt, styles, obstacle_keys = generate_iris_regions(
35     bounding_lb,
36     bounding_ub,
37     endpoints,
38     num_regions,
39     obstacle_map,
40 )
41
42 # Problem initialization
43 positions, region_assignments, var_dict = initialize_footstep_problem(
44     len(iris_regions),
45     steps,
46 )
47
48 # Constraints
49 constraints = []
50 constraints += add_region_membership_constraints(
51     positions,
52     region_assignments,
53     iris_regions,
54     big_M,
55 )
56 constraints += add_reachability_constraints(positions)
57 constraints += add_trajectory_endpoint_constraints(
58     positions,
59     [start_left, start_right],
60     goal_pos,
61 )
62
63 # Objective and solve
64 objective = add_objective(["feasibility"])
65 status, cost, solution = solve_problem(
66     constraints,
67     objective,
68     var_dict,
69 )

```

Finger Selection

Input instruction.

To stably grasp an object with four fingers, place each finger at well-separated contact points around the object perimeter so that the contacts distribute force symmetrically and improve grasp robustness. The friction coefficient is $\mu = 1.0$.

Generated executable program.

```
1 # Object geometry and task setup
2 vertices_list = [
3     [-0.3, 0.9],
4     [-1.3, -1.0],
5     [0.9, -1.0],
6     [1.2, -0.3],
7     [0.4, 0.3],
8 ]
9
10 n_fingers = 4
11 friction_coef = 1.0
12 delta = 0.1
13 delta_n = 0.0
14
15 # Object construction and contact sampling
16 obj = build_finger_select_object(vertices_list)
17 sampled_points, sampled_normals = sample_finger_contact_candidates(
18     obj,
19     delta,
20     delta_n,
21 )
22
23 # Problem initialization
24 selection, radius, var_dict = initialize_finger_selection_problem(
25     n_fingers,
26     len(sampled_points),
27 )
28
29 # Constraints
30 constraints = []
31 constraints += add_contact_selection_constraints(selection)
32 constraints += add_wrench_constraints(
33     sampled_points,
34     sampled_normals,
35     friction_coef,
36 )
37
38 # Objective and solve
39 objective = add_objective(["feasibility"])
40 status, cost, solution = solve_problem(
41     constraints,
42     objective,
43     var_dict,
44 )
```

Multi-UAV Planning

Input instruction.

Two UAVs begin from different locations in the workspace. UAV 1 starts at $(2.4, -1.8)$ and should first pass around the right side of the lower obstacle before moving through the open corridor. UAV 2 starts at $(-2.3, 1.7)$ and should pass around the left side of the upper obstacle before entering the same open corridor. The two UAVs must complete a rendezvous task by arriving at the same collision-free meeting location while avoiding all obstacles and maintaining smooth trajectories within the bounded workspace $[-3.0, 3.0] \times [-3.0, 3.0]$.

Generated executable program.

```
1 # Environment and task setup
2 bounding_lb = [-3.0, -3.0]
3 bounding_ub = [3.0, 3.0]
4
```

```

5 start_pos_uav1 = [2.4, -1.8]
6 start_pos_uav2 = [-2.3, 1.7]
7 shared_meet_pos = [0.3, 0.2]
8
9 goal_pos_uav1 = shared_meet_pos
10 goal_pos_uav2 = shared_meet_pos
11
12 num_segments = 12
13 num_regions = 32
14 big_M = 10.0
15 side_width = 0.2
16
17 obstacle_map = {
18     "obst1": {
19         "lb": [[0.5], [-2.0]],
20         "ub": [[1.4], [-0.8]],
21         "style": "right",
22     },
23     "obst2": {
24         "lb": [[-1.5], [0.7]],
25         "ub": [[-0.5], [1.8]],
26         "style": "left",
27     },
28     "obst3": {
29         "lb": [[-0.4], [-0.6]],
30         "ub": [[0.6], [0.0]],
31         "style": "none",
32     },
33 }
34
35 # -----
36 # UAV 1: single-UAV planning sequence
37 # -----
38 endpoints_uav1 = [start_pos_uav1, goal_pos_uav1]
39 iris_regions_uav1, obstacles_list_plt_uav1, styles_uav1, obstacle_keys_uav1 =
40     generate_iris_regions(
41         bounding_lb,
42         bounding_ub,
43         endpoints_uav1,
44         num_regions,
45         obstacle_map,
46     )
47 iris_regions_uav1, styles_uav1 = augment_iris_regions(
48     iris_regions_uav1,
49     obstacle_map,
50     styles_uav1,
51     side_width,
52 )
53
54 C_uav1, H_uav1, var_dict_uav1 = initialize_trajectory_problem(
55     num_segments,
56     len(iris_regions_uav1),
57 )
58
59 constraints_uav1 = []
60 constraints_uav1 += add_segment_assignment_constraints(H_uav1)
61 constraints_uav1 += add_region_membership_constraints(
62     C_uav1,
63     H_uav1,
64     iris_regions_uav1,
65     big_M,
66 )
67 constraints_uav1 += add_spatial_relation_constraints(H_uav1, styles_uav1)
68 constraints_uav1 += add_trajectory_continuity_constraints(C_uav1)
69 constraints_uav1 += add_trajectory_endpoint_constraints(
70     C_uav1,
71     start_pos_uav1,
72     goal_pos_uav1,
73 )
74 objective_uav1 = add_objective(["trajectory_smoothness"])
75 status, cost, solution = solve_problem(
76     constraints_uav1,
77     objective_uav1,
78     var_dict_uav1,
79 )
80 # -----
81 # UAV 2: single-UAV planning sequence
82 # -----

```

```

83 endpoints_uav2 = [start_pos_uav2, goal_pos_uav2]
84 iris_regions_uav2, obstacles_list_plt_uav2, styles_uav2, obstacle_keys_uav2 =
    generate_iris_regions(
85     bounding_lb,
86     bounding_ub,
87     endpoints_uav2,
88     num_regions,
89     obstacle_map,
90 )
91
92 iris_regions_uav2, styles_uav2 = augment_iris_regions(
93     iris_regions_uav2,
94     obstacle_map,
95     styles_uav2,
96     side_width,
97 )
98
99 C_uav2, H_uav2, var_dict_uav2 = initialize_trajectory_problem(
100     num_segments,
101     len(iris_regions_uav2),
102 )
103
104 constraints_uav2 = []
105 constraints_uav2 += add_segment_assignment_constraints(H_uav2)
106 constraints_uav2 += add_region_membership_constraints(
107     C_uav2,
108     H_uav2,
109     iris_regions_uav2,
110     big_M,
111 )
112 constraints_uav2 += add_spatial_relation_constraints(H_uav2, styles_uav2)
113 constraints_uav2 += add_trajectory_continuity_constraints(C_uav2)
114 constraints_uav2 += add_trajectory_endpoint_constraints(
115     C_uav2,
116     start_pos_uav2,
117     goal_pos_uav2,
118 )
119
120 objective_uav2 = add_objective(["trajectory_smoothness"])
121 status, cost, solution = solve_problem(
122     constraints_uav2,
123     objective_uav2,
124     var_dict_uav2,
125 )

```

UAV-IK Planning

Input instruction.

A UAV starts from $(2.2, -2.2)$ and flies through a cluttered workspace to support a downstream manipulation task. It should pass around the right side of the lower obstacle and avoid the central obstacle before reaching a collision-free handoff location. From this handoff location, a robotic arm with four links must reach the manipulation target while keeping its links collision-free. The complete plan must jointly coordinate the UAV trajectory and the arm configuration within the bounded workspace $[-3.0, 3.0] \times [-3.0, 3.0]$.

Generated executable program.

```

1 # Environment and task setup
2 bounding_lb = [-3.0, -3.0]
3 bounding_ub = [3.0, 3.0]
4
5 uav_start = [2.2, -2.2]
6 handoff_seed = [0.2, 0.4]
7 ik_target = [-1.2, 1.5]
8
9 num_segments = 12
10 num_regions = 30
11 big_M = 10.0
12 side_width = 0.2
13
14 link_lengths = [0.8, 0.7, 0.5, 0.4]
15 n_poly = 8
16 num_intermediate = 10

```

```

17
18 obstacle_map = {
19     "obst1": {
20         "lb": [[0.6], [-1.8]],
21         "ub": [[1.4], [-0.8]],
22         "style": "right",
23     },
24     "obst2": {
25         "lb": [[-0.5], [-0.4]],
26         "ub": [[0.5], [0.5]],
27         "style": "none",
28     },
29     "obst3": {
30         "lb": [[-1.8], [0.8]],
31         "ub": [[-0.8], [1.6]],
32         "style": "none",
33     },
34 }
35
36 shared_meet_pos=[0.3,0.2]
37
38 # Region generation
39 endpoints = [uav_start, handoff_seed, ik_target]
40 iris_regions, obstacles_list_plt, styles, obstacle_keys = generate_iris_regions(
41     bounding_lb,
42     bounding_ub,
43     endpoints,
44     num_regions,
45     obstacle_map,
46 )
47
48 iris_regions, styles = augment_iris_regions(
49     iris_regions,
50     obstacle_map,
51     styles,
52     side_width,
53 )
54
55 # -----
56 # UAV trajectory planning
57 # -----
58 C_uav, H_uav, var_dict_uav = initialize_trajectory_problem(
59     num_segments,
60     len(iris_regions),
61 )
62
63 constraints_uav = []
64 constraints_uav += add_segment_assignment_constraints(H_uav)
65 constraints_uav += add_region_membership_constraints(
66     C_uav,
67     H_uav,
68     iris_regions,
69     big_M,
70 )
71 constraints_uav += add_spatial_relation_constraints(H_uav, styles)
72 constraints_uav += add_trajectory_continuity_constraints(C_uav)
73 constraints_uav += add_trajectory_endpoint_constraints(
74     C_uav,
75     uav_start,
76     shared_meet_pos,
77 )
78
79 objective_uav = add_objective(["trajectory_smoothness"])
80 status, cost, solution = solve_problem(
81     constraints_uav,
82     objective_uav,
83     var_dict_uav,
84 )
85
86 # -----
87 # IK planning from the shared UAV handoff location
88 # -----
89 pivots, rotations, collision_points, var_dict_ik = initialize_kinematic_problem(
90     link_lengths,
91     n_poly,
92     num_intermediate,
93     shared_meet_pos,
94 )
95 constraints_ik=[]

```

```

96 constraints_ik += add_base_target_constraints(
97     pivots[0],
98     pivots[-1],
99     shared_meet_pos,
100     ik_target,
101 )
102 constraints_ik += add_kinematic_constraints(
103     pivots,
104     rotations,
105     link_lengths,
106 )
107 constraints_ik += add_collision_free_constraints(
108     collision_points,
109     iris_regions,
110 )
111
112 # Objective and solve
113 objective = add_objective([
114     "feasibility",
115 ])
116
117 status, cost, solution = solve_problem(
118     constraints_ik,
119     objective,
120     var_dict_ik,
121 )

```

IK-Footstep Planning

Input instruction.

A biped robot starts with its left foot at $(-3.0, -2.6)$ and right foot at $(-2.5, -2.6)$. It must walk through the workspace while avoiding obstacles and stop at a feasible manipulation stance. From this stance, its arm with four links must reach a target point near the upper-right workspace region while keeping the articulated chain collision-free. The locomotion and arm motion must be coordinated through a shared base location within the bounded workspace $[-4.0, 4.0] \times [-4.0, 4.0]$.

Generated executable program.

```

1 # Environment and task setup
2 bounding_lb = [-4.0, -4.0]
3 bounding_ub = [4.0, 4.0]
4
5 start_left = [-3.0, -2.6]
6 start_right = [-2.5, -2.6]
7 ik_target = [1.3, 1.1]
8
9 steps = 12
10 step_dist = 1.0
11 num_regions = 24
12 big_M = 10.0
13 side_width = 0.2
14
15 link_lengths = [0.8, 0.7, 0.5, 0.4]
16 n_poly = 8
17 num_intermediate = 10
18
19 obstacle_map = {
20     "obst1": {
21         "lb": [-1.8, -1.4],
22         "ub": [-0.7, -0.3],
23         "style": "none",
24     },
25     "obst2": {
26         "lb": [0.5, -1.2],
27         "ub": [1.6, -0.2],
28         "style": "none",
29     },
30     "obst3": {
31         "lb": [-0.4, 0.6],
32         "ub": [0.6, 1.5],
33         "style": "none",
34     },
35 }

```

```

36
37 shared_base = [0.3, 0.2]
38
39 # Region generation
40 endpoints = [start_left, start_right, shared_base, ik_target]
41 iris_regions, obstacles_list_plt, styles, obstacle_keys = generate_iris_regions(
42     bounding_lb,
43     bounding_ub,
44     endpoints,
45     num_regions,
46     obstacle_map,
47 )
48
49 iris_regions, styles = augment_iris_regions(
50     iris_regions,
51     obstacle_map,
52     styles,
53     side_width,
54 )
55
56 # -----
57 # Footstep planning to the shared manipulation stance
58 # -----
59 foot_positions, H_foot, var_dict_foot = initialize_footstep_problem(
60     len(iris_regions),
61     steps,
62 )
63
64 constraints = []
65 constraints += add_region_membership_constraints(
66     foot_positions,
67     H_foot,
68     iris_regions,
69     big_M,
70 )
71 constraints += add_reachability_constraints(foot_positions)
72 constraints += add_trajectory_endpoint_constraints(
73     foot_positions,
74     [start_left, start_right],
75     shared_base,
76 )
77
78 # Objective and solve
79 objective = add_objective(["feasibility"])
80 status, cost, solution = solve_problem(
81     constraints,
82     objective,
83     var_dict_foot,
84 )
85
86 # -----
87 # IK planning from the shared stance
88 # -----
89 pivots, rotations, collision_points, var_dict_ik = initialize_kinematic_problem(
90     link_lengths,
91     n_poly,
92     num_intermediate,
93     shared_base,
94 )
95 constraints_ik = []
96 constraints_ik += add_base_target_constraints(
97     pivots[0],
98     pivots[-1],
99     shared_base,
100     ik_target,
101 )
102 constraints_ik += add_kinematic_constraints(
103     pivots,
104     rotations,
105     link_lengths,
106 )
107 constraints_ik += add_collision_free_constraints(
108     collision_points,
109     iris_regions,
110 )
111
112 # Objective and solve
113 objective = add_objective(["feasibility"])
114

```

```

115 status, cost, solution = solve_problem(
116     constraints_ik,
117     objective,
118     var_dict_ik,
119 )

```

K LLM Prompt

This section presents the detailed system prompt used in our method, together with a representative model output. In the multi-UAV example, the fine-tuned LLM directly constructs decision variables and constraints for the shared rendezvous point, rather than decomposing the task into two independent single-UAV problems with a manually predicted meeting location. This demonstrates that the model learns to discretize multimodal robot planning tasks into solver-grounded MIP formulations.

LLM System Prompt

Input instruction.

You are an expert in multi-modal mixed-integer programming (MIP) for robotic planning. Read the natural-language mission instruction, the optimal OBJECT.GEOMETRY block, and the ENVIRONMENT block. Identify the correct modality and write a runnable Python program that solves the problem. You are provided with an API handbook containing MIP discretization functions. You may use the functions listed in the API handbook below, as well as functions from the cvxpy package. Note that `obstacle_map` entries use the standard axis-aligned box form: `{"lb": [...], "ub": [...], "style": ...}`.

Your output must contain a `<think>...</think>` block that concisely explains the planning rationale, followed by a `<code>...</code>` block containing the complete executable Python code.

Follow the structure below:

1. build object geometry or safe regions when the modality requires them,
2. initialize decision variables,
3. add constraints stage by stage,
4. call the modality solver.

Keep all arguments explicit, and write the complete executable Python code in the correct format.

Multi-UAV Rendezvous Planning

Input instruction.

Two UAVs operate in the bounded workspace $[-3.0, 3.0] \times [-3.0, 3.0]$. UAV 1 starts at $(-2.0, -2.0)$ and UAV 2 starts at $(2.0, 2.0)$. Because of communication-range limits, neither UAV should travel all the way to the other station. Instead, they must find an intermediate collision-free shared meeting point and plan smooth trajectories to this rendezvous location. The environment contains one rectangular obstacle with lower bound $(0.223, 0.729)$ and upper bound $(0.765, 1.377)$, and the route should approach this obstacle from the left side before continuing toward the rendezvous.

Generated executable program.

```

1 <think>
2 This is a two-UAV rendezvous problem. I need one shared meeting-point variable,
  then each UAV should plan from its own station to that same point. I will
  generate shared safe regions, build one trajectory problem per UAV, enforce
  region assignment, obstacle avoidance, spatial style constraints, continuity,
  and start-goal constraints. Finally, I will explicitly constrain the shared
  meeting point to lie in one collision-free safe region.
3 </think>
4
5 <code>
6

```

```

7 import cvxpy as cp
8 import numpy as np
9
10 num_uavs = 2
11 num_segments_list = [12, 12]
12
13 bounding_lb = np.array([-3.0, -3.0], dtype=float)
14 bounding_ub = np.array([3.0, 3.0], dtype=float)
15
16 start_1 = np.array([-2.0, -2.0], dtype=float)
17 goal_last = np.array([2.0, 2.0], dtype=float)
18
19 num_regions = 24
20 big_M = 10.0
21 side_width = 0.2
22
23 obstacle_map = {
24     "obst1": {
25         "lb": np.array([[0.223], [0.729]], dtype=float),
26         "ub": np.array([[0.765], [1.377]], dtype=float),
27         "style": "left",
28     },
29 }
30
31 # -----
32 # Shared safe-region generation
33 # -----
34 safe_regions, obstacles_list_plt, styles, obstacle_keys =
35     generate_iris_regions(bounding_lb, bounding_ub, start_1, goal_last,
36                           num_regions, obstacle_map)
37
38 safe_regions, styles_related_regions =
39     augment_iris_regions(safe_regions, obstacle_map, styles, side_width)
40
41 num_safe_regions = len(safe_regions)
42
43 # -----
44 # Multi-UAV trajectory variables
45 # -----
46 C_list, H_list, var_dict =
47     initialize_trajectory_problem(num_segments_list, num_safe_regions)
48
49 # -----
50 # LLM generated shared point constraints
51 # -----
52 shared_points_list = []
53 for i in range(max(0, num_uavs - 1)):
54     shared_point = cp.Variable(2, name=f"shared_points_{i}")
55     var_dict[f"shared_points_{i}"] = shared_point
56     shared_points_list.append(shared_point)
57
58 # -----
59 # Per-UAV planning constraints
60 # -----
61 constraints = []
62
63 for uav_idx in range(num_uavs):
64     C = C_list[uav_idx]
65     H = H_list[uav_idx]
66     num_segments = num_segments_list[uav_idx]
67
68     constraints += add_segment_assignment_constraints(H)
69
70     constraints += add_spatial_relation_constraints(H, styles)
71
72     constraints += add_region_membership_constraints(C, H, safe_regions, big_M=big_M)
73
74     constraints += add_trajectory_continuity_constraints(C)
75
76     constraints += add_trajectory_endpoint_constraints(C, start_pos, goal_pos)
77
78 # -----
79 # Shared rendezvous feasibility constraints
80 # -----
81 for sp_idx, shared_point in enumerate(shared_points_list):
82     num_r = len(safe_regions)
83
84     sp_z = cp.Variable(num_r, boolean=True, name=f"z_shared_point_{sp_idx}")
85     var_dict[f"z_shared_point_{sp_idx}"] = sp_z

```

```

83
84     sp_y_vars = [
85         cp.Variable(2, name=f"y_shared_point_{sp_idx}_{r}")
86         for r in range(num_r)
87     ]
88
89     for r in range(num_r):
90         var_dict[f"y_shared_point_{sp_idx}_{r}"] = sp_y_vars[r]
91
92     constraints += [
93         shared_point[0] == sum(y[0] for y in sp_y_vars),
94         shared_point[1] == sum(y[1] for y in sp_y_vars),
95         cp.sum(sp_z) == 1,
96     ]
97
98     for r in range(num_r):
99         A_r = safe_regions[r].A()
100         b_r = safe_regions[r].b().flatten()
101
102         for j in range(A_r.shape[0]):
103             constraints.append(
104                 A_r[j, 0] * sp_y_vars[r][0]
105                 + A_r[j, 1] * sp_y_vars[r][1]
106                 <= b_r[j] * sp_z[r]
107             )
108     objective = add_objective(["trajectory_smoothness"])
109     # -----
110     # Solve
111     # -----
112     status, cost, H_vals, C_vals = solve_problem(constraints, objective, var_dict)
113 </code>

```